

지휘 시뮬레이션 게임 🎵

팀명 : 마에스트로

팀장 : 이영현

팀원 : 유지민, 이민, 이승열, 조정민, 지승배, 한승훈

발표일 : 2026.09.08



대한상공회의소

🎵 CONTENTS

01 프로젝트 개요

02 시스템 설계

03 기능 구현

04 UVM 검증

05 구현 결과

06 트러블 슈팅

07 느낀 점

01

프로젝트 개요

🎵 프로젝트 배경 및 목표

■ 프로젝트 배경

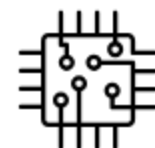
카메라 영상 기반의 사용자 동작 인식을 FPGA에서 실시간으로 처리하여 음악과 상호작용하는 지휘 게임 구현

■ 프로젝트 목표

01 실시간 영상 처리



OV7670 영상 입력



FPGA 영상 처리



VGA 실시간 출력

02 지휘 동작 인식



RED 지휘봉의 X·Y 좌표 추적
→ 지휘 패턴 및 BPM 계산



GREEN 손의 Y좌표 변화 이용
→ 볼륨 계산

03 통신 기반 음악 제어



FPGA 제어 데이터 생성



UART로 PC 전달



음악 속도·볼륨 반영

♪ 개발 환경



Basys3
FPGA Board



OV7670
Camera Module



Vivado / VCS / Verdi
RTL Design
& Simulation

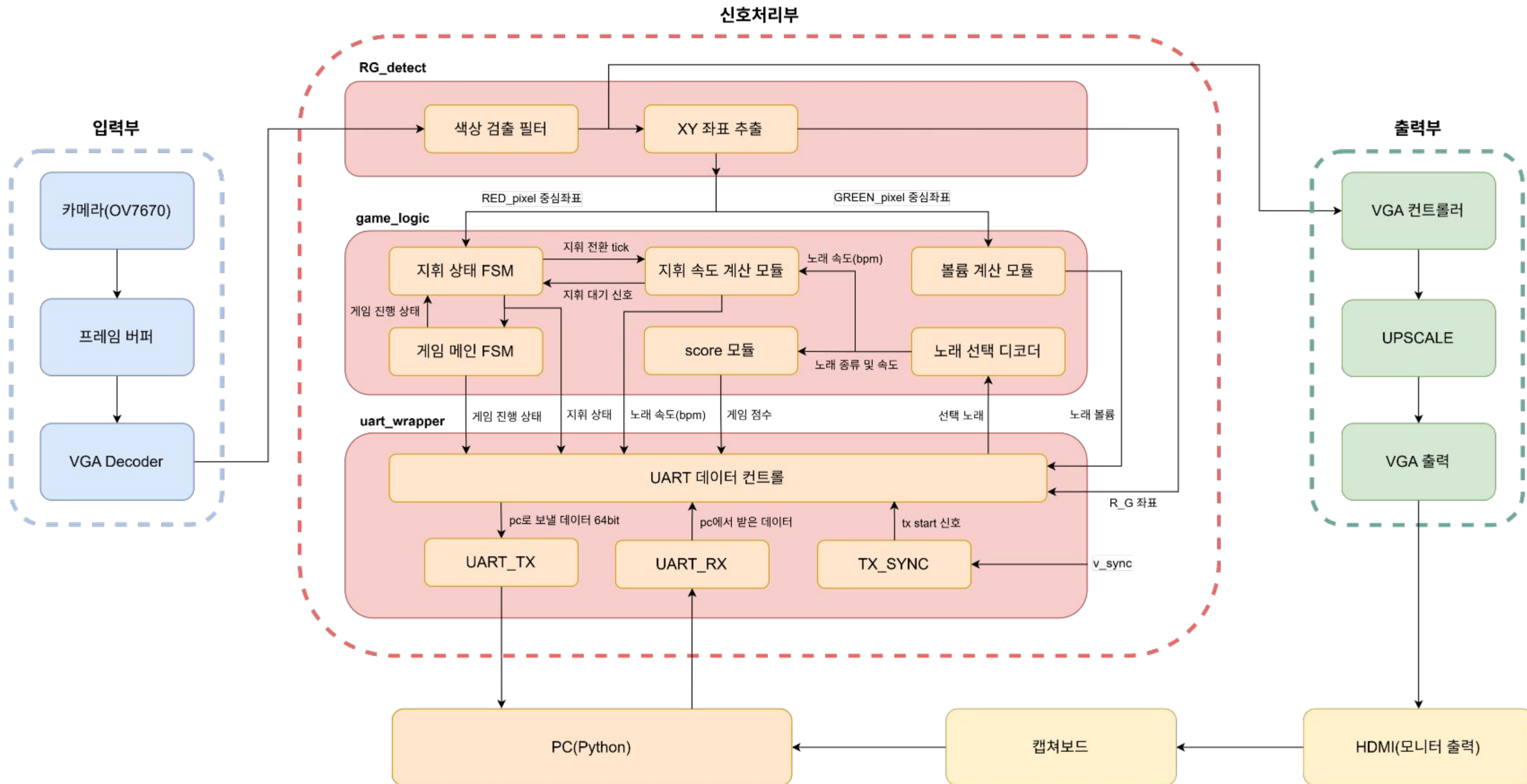


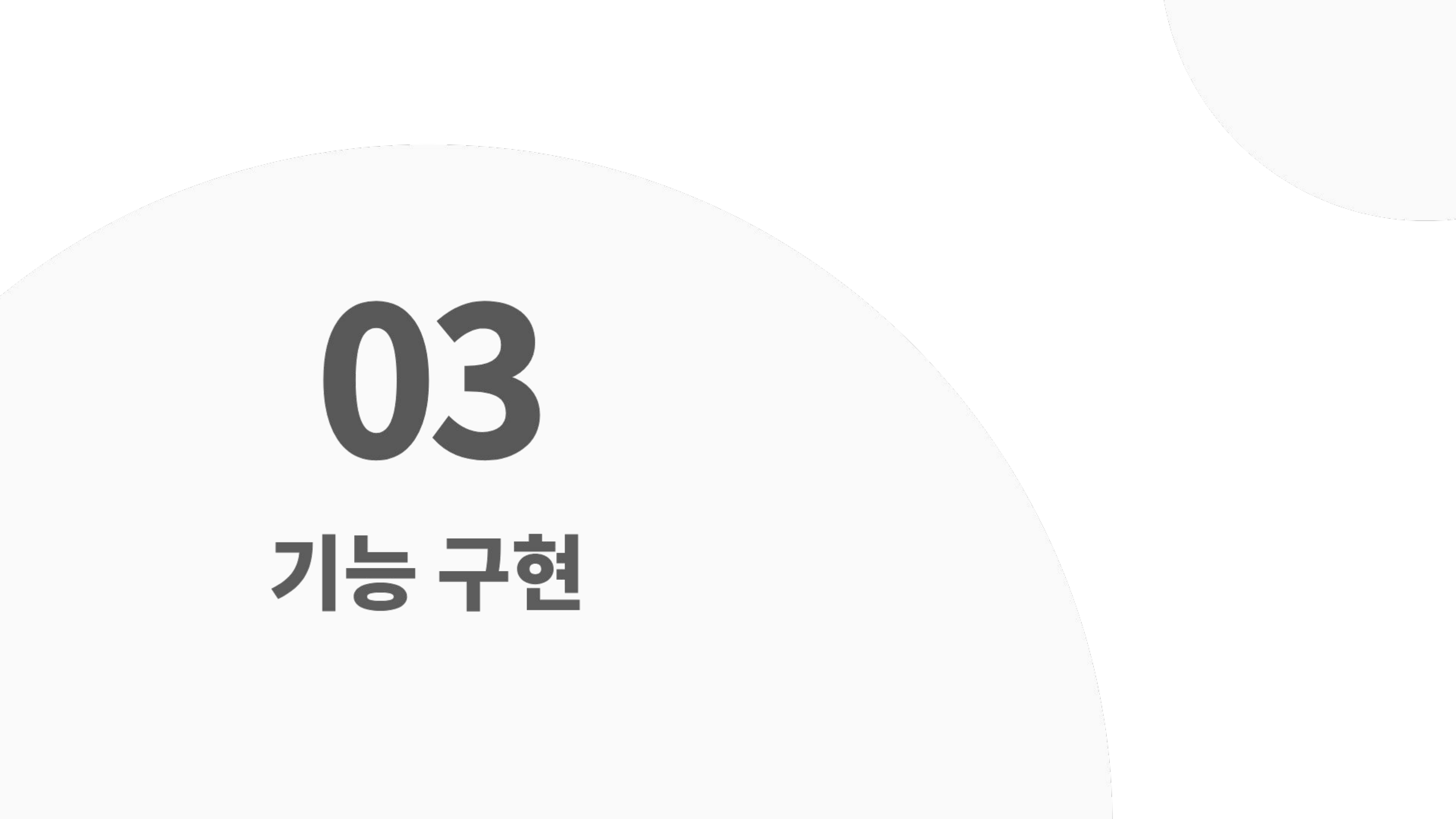
Python
PC Application

02

시스템 설계

🎵 전체 시스템 아키텍처



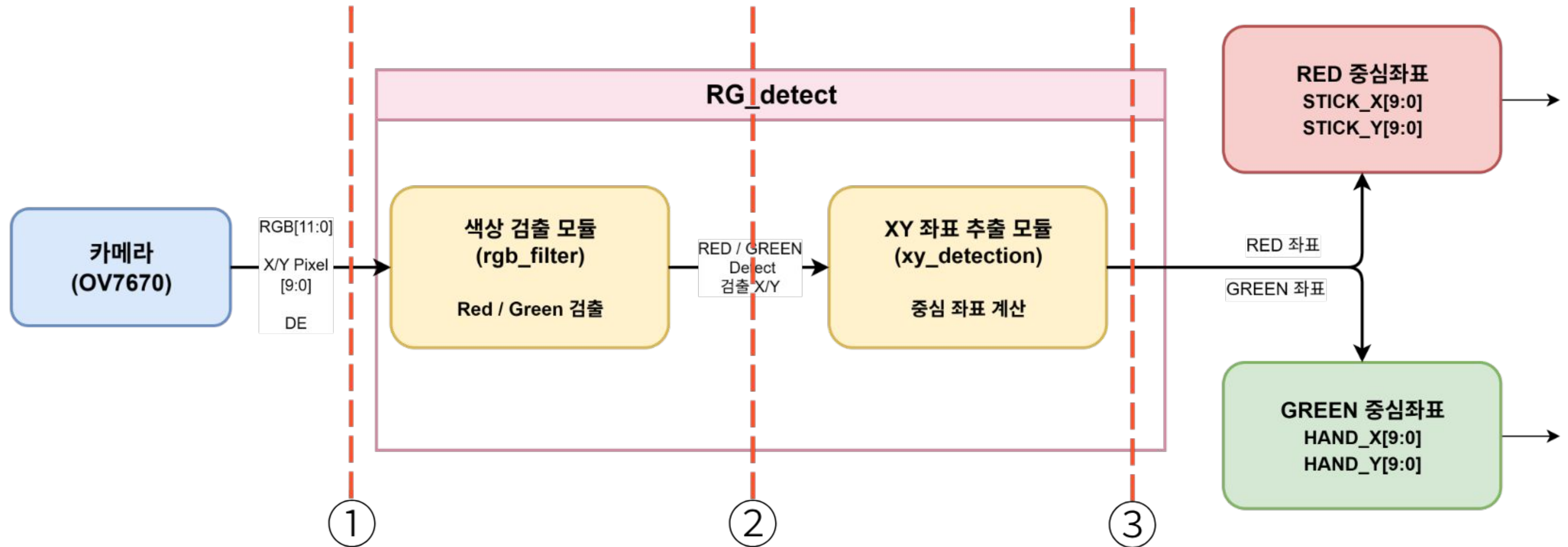


03

기능 구현

🎵 입력부 | RG Detect

■ 카메라로부터 입력된 영상에서 RED와 GREEN 색상을 검출하고, 각 색상의 중심 좌표를 추출하는 모듈



① 영상 데이터 입력

OV7670 카메라 RGB 영상 수신
/ 픽셀 좌표 및 DE 신호 전달

② RED / GREEN 색상 검출

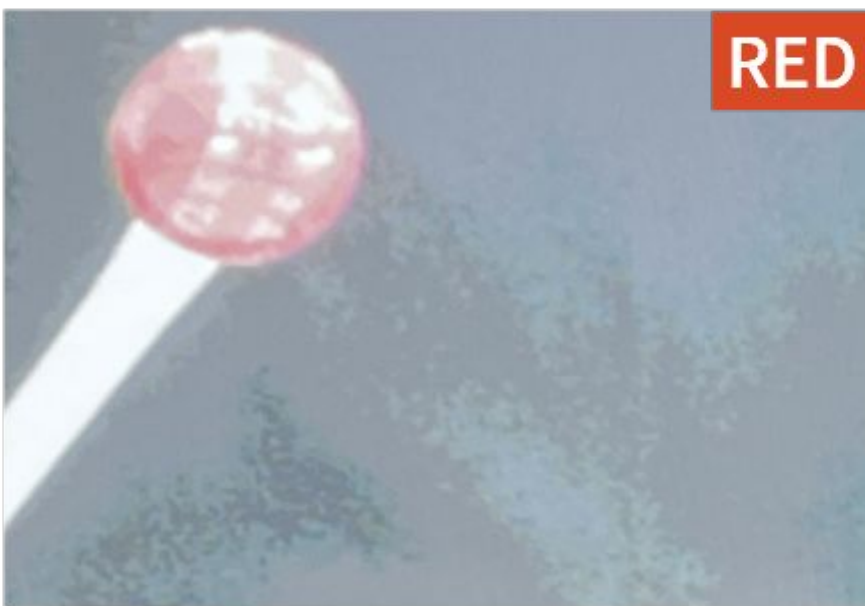
rgb_filter 기반 색상 영역 판별
/ RED·GREEN 픽셀 분리

③ 중심 좌표 추출

xy_detection으로 색상 영역 좌표 누적
/ Bounding 영역 기반 중심 좌표 계산

🎵 기능 구현 01 Red, Green 색상 검출 모듈

01 Camera Input

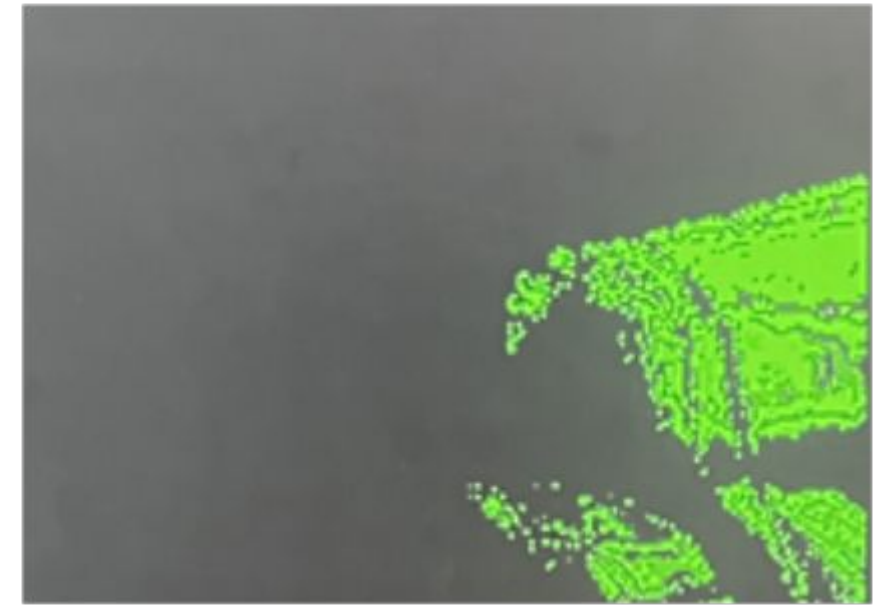


02 RGB 비교 조건

```
1 if ((red >= RED_MIN) &&  
2     (red >= green) &&  
3     ((red - green) >= RED_G_MARGIN) &&  
4     (red >= blue) &&  
5     ((red - blue) >= RED_B_MARGIN)) begin  
6  
7     red_detect = 1'b1;  
8 end
```

```
1 if ((green >= GREEN_MIN) &&  
2     (green >= red) &&  
3     ((green - red) >= GREEN_R_MARGIN) &&  
4     (green >= blue) &&  
5     ((green - blue) >= GREEN_B_MARGIN)) begin  
6  
7     green_detect = 1'b1;  
8 end
```

03 Detection Result

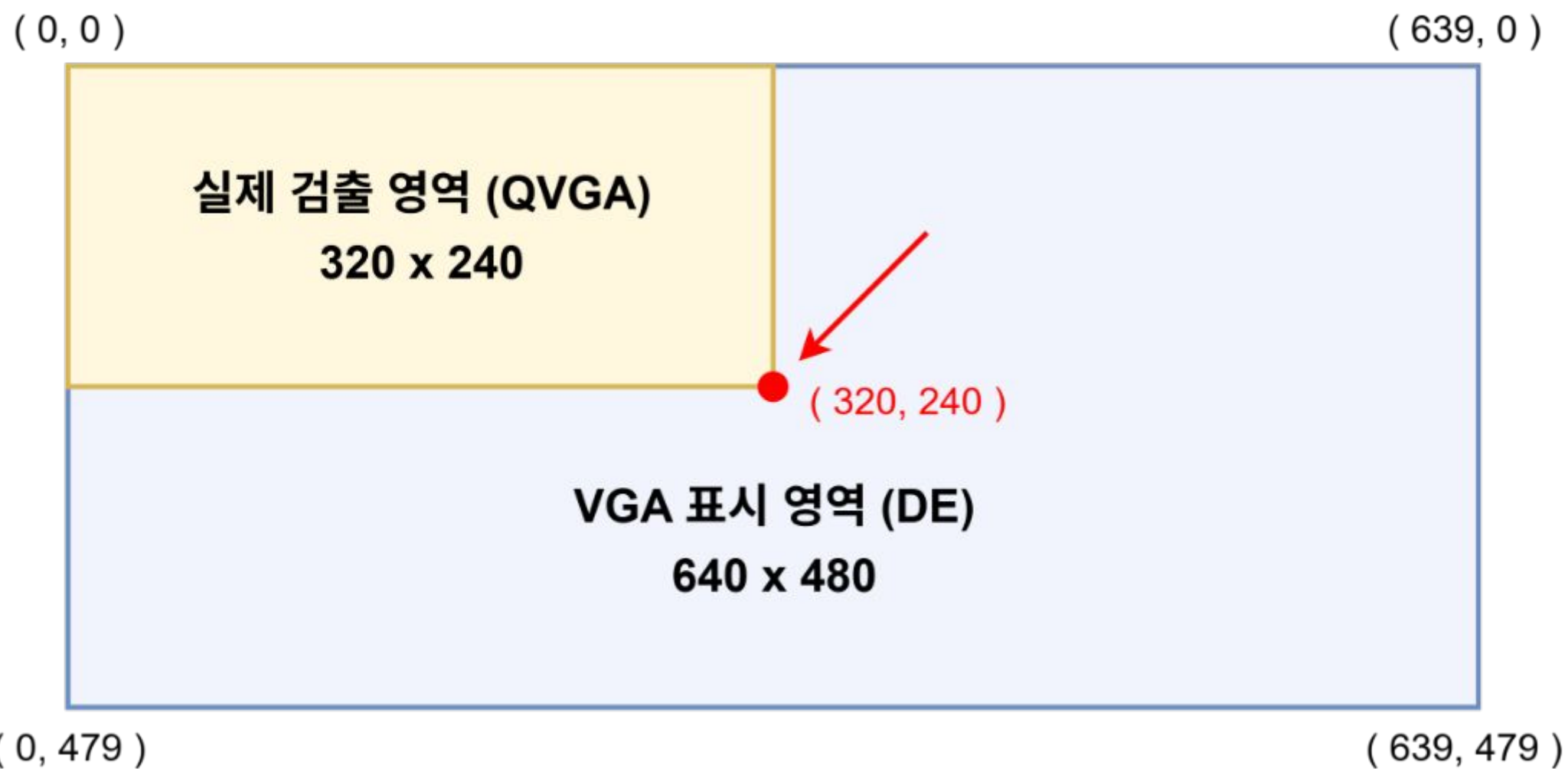


🎵 기능 구현 02 Red, Green 좌표 추출 모듈

01 RED 검출 결과



02 게임 화면 반영 예시

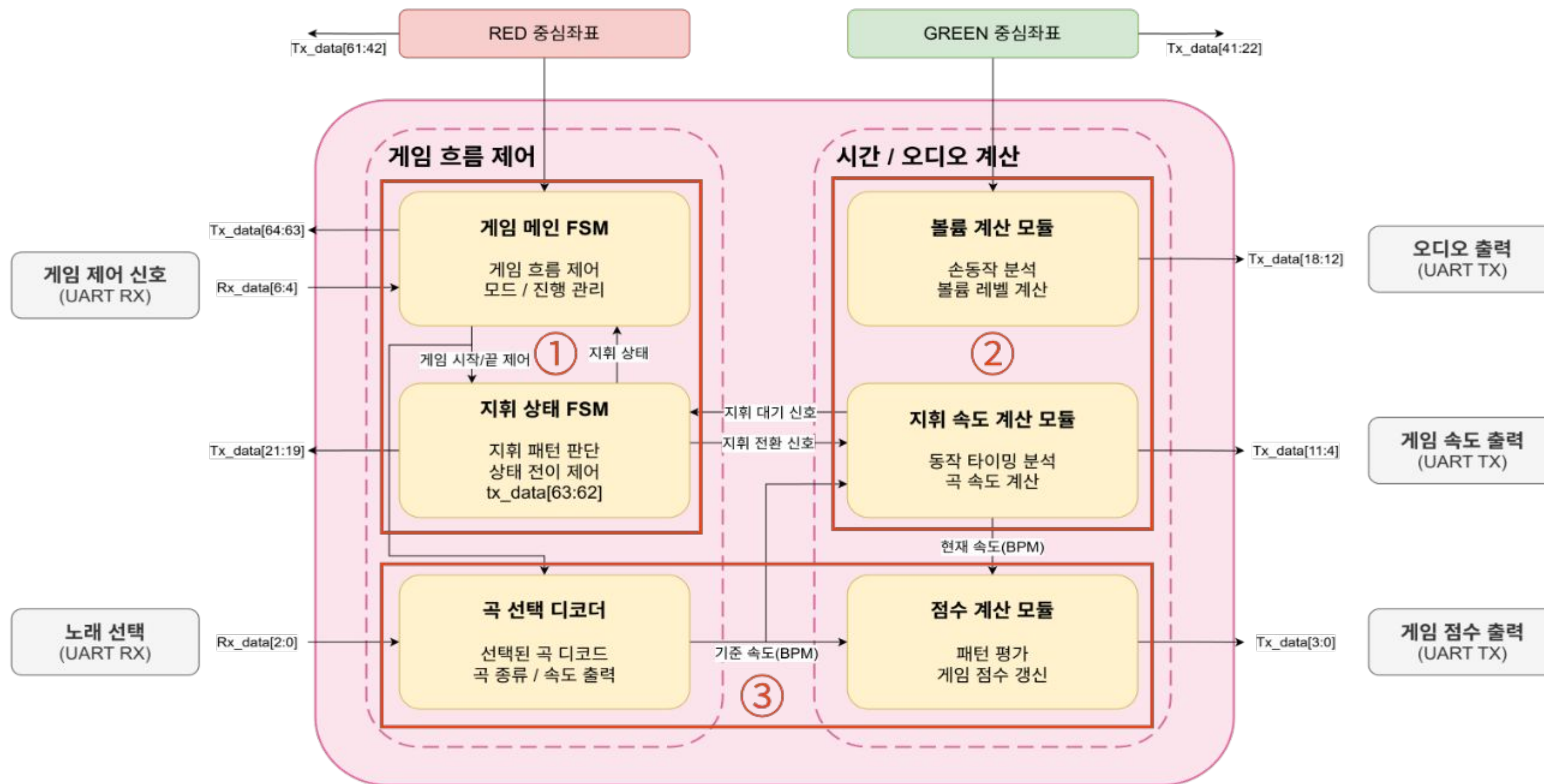


■ 핵심 동작 흐름

- ① QVGA(320×240) 한 프레임 동안 RED/GREEN 픽셀 좌표 누적
- ② 프레임 종료 시 frame_done 기반 내부 o_vsync를 생성
- ③ 계산된 중심 좌표를 좌표 유효 시점에 맞춰 후단 모듈로 전달

🎵 제어부 | Game Logic

■ RED / GREEN 중심 좌표를 기반으로 게임 진행, 지휘 상태, 속도·볼륨, 점수를 계산하는 제어 모듈



① 게임 흐름 제어

게임 메인 FSM, 지휘 상태 FSM을 통해 모드 및 진행 상태 관리

② 속도 / 볼륨 계산

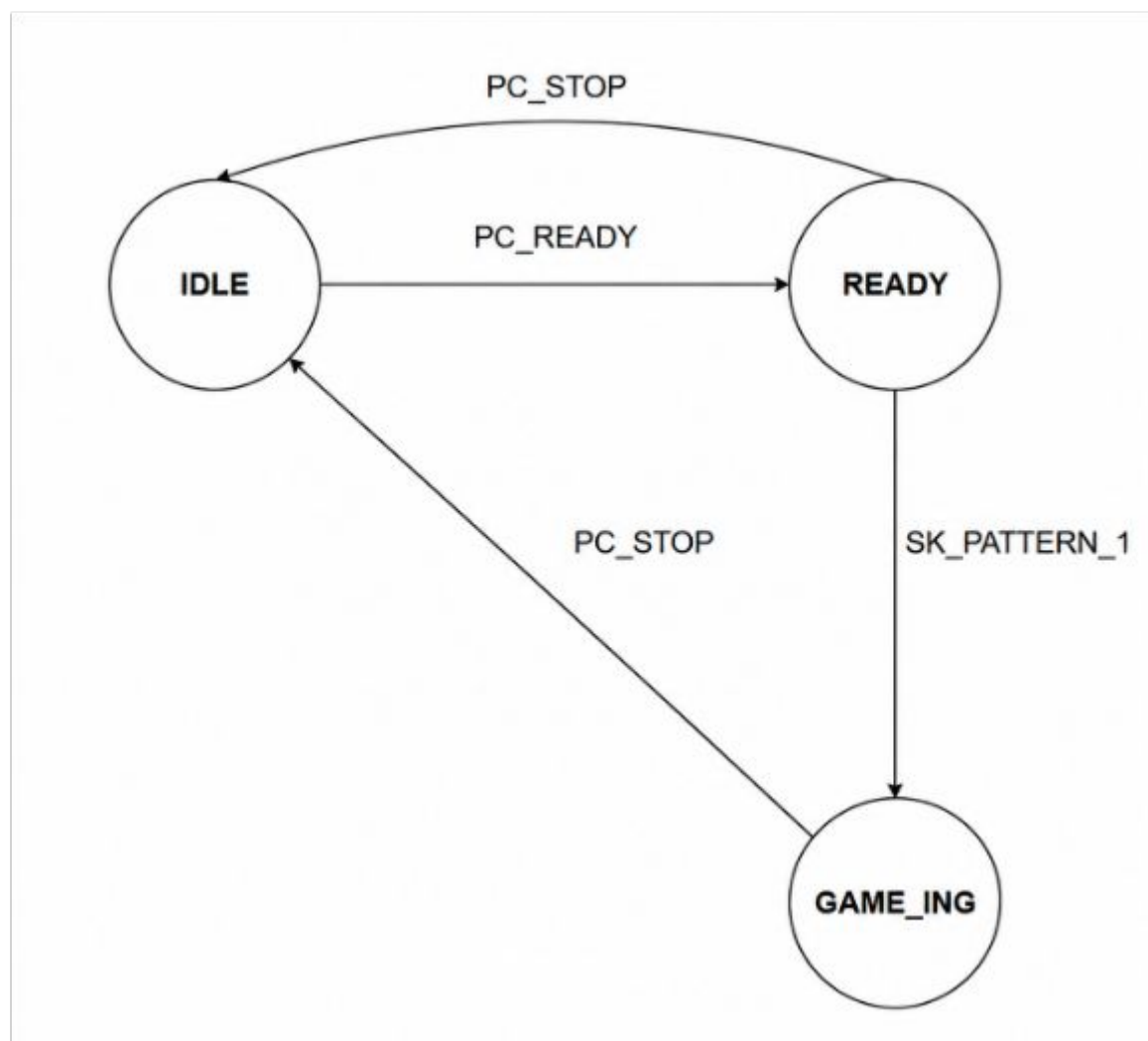
RED 좌표 기반 속도 계산 / GREEN 좌표 기반 볼륨 계산

③ 곡 선택 및 점수 계산

선택 곡 기준 속도 반영 / 패턴 평가를 통한 점수 갱신

🎵 기능 구현 03 게임 메인 제어 모듈

■ FSM



STATE	역할	핵심 동작
IDLE	초기화	PC_READY 받으면 Start 신호 출력
READY	시스템 시작 대기	Pattern_1 진입 시 Gaming 시작
GAME_ING	게임 상태	게임 동작 중 PC_STOP 들어오면 done

■ 개요

PC UI 상태를 입력 받아 게임의 시작과 종료를 제어하는 전체 게임 흐름을 관리하는 모듈

■ 주요 특징

01 PC UI 연동

PC_READY / PC_STOP 신호로 게임 시작·종료 제어

02 게임 상태 관리

시작 대기 → 게임 진행 → 종료까지 전체 흐름 제어

🎵 기능 구현 04 지휘 상태 판정 모듈

■ 개요

지휘봉 중심 좌표(X, Y)를 입력받아 설정된 ZONE을 기반으로 위치를 판별하고, 지휘 상태를 출력

■ 주요 특징

01

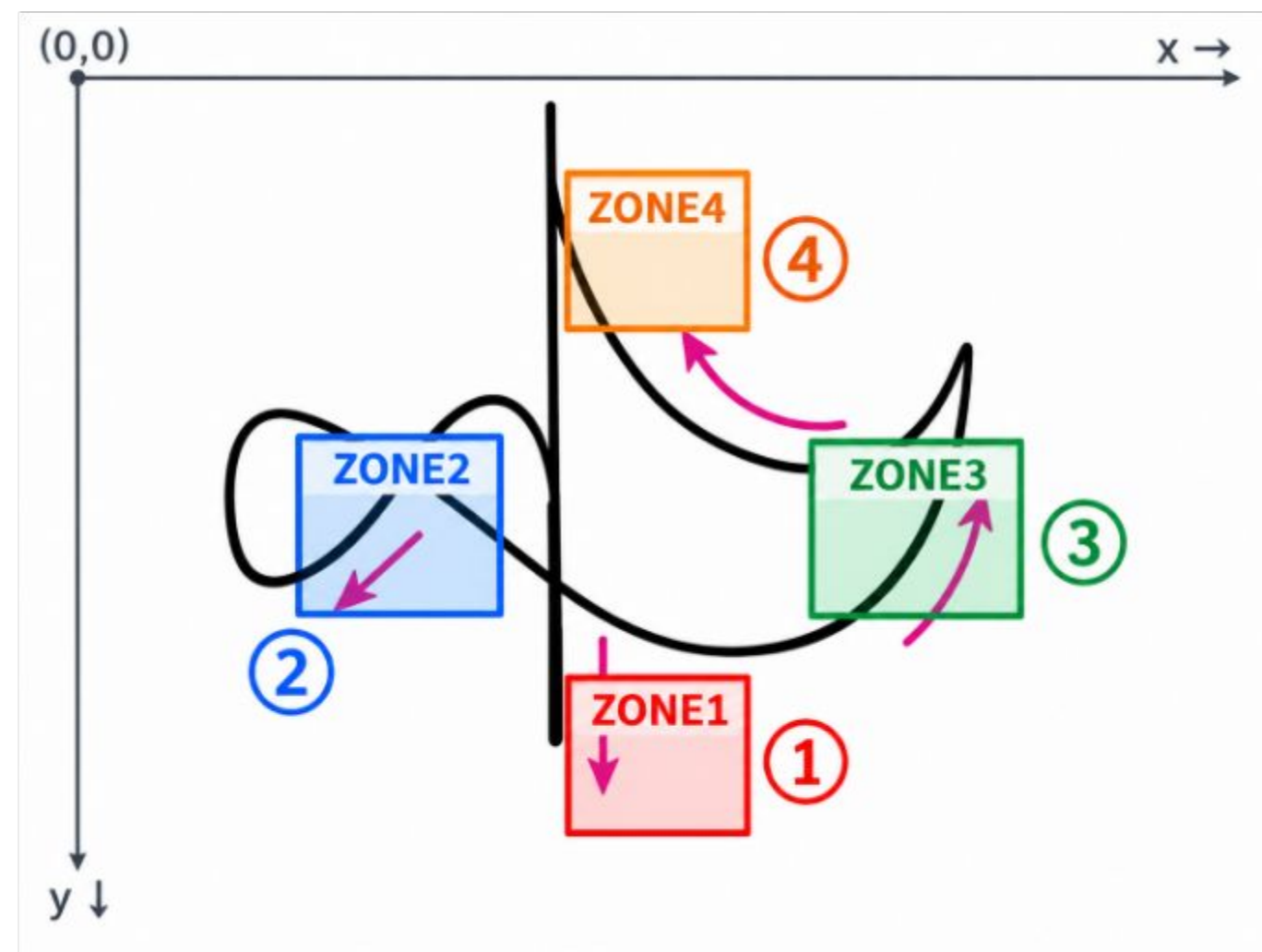
ZONE 기반 상태 전이
지휘봉이 해당 ZONE에 HIT 시 다음 상태로 전이

02

게임 시작 조건
지휘봉이 ZONE4에 위치하면 READY 상태에 대기,
ZONE1로 내려가면 음악 시작

03

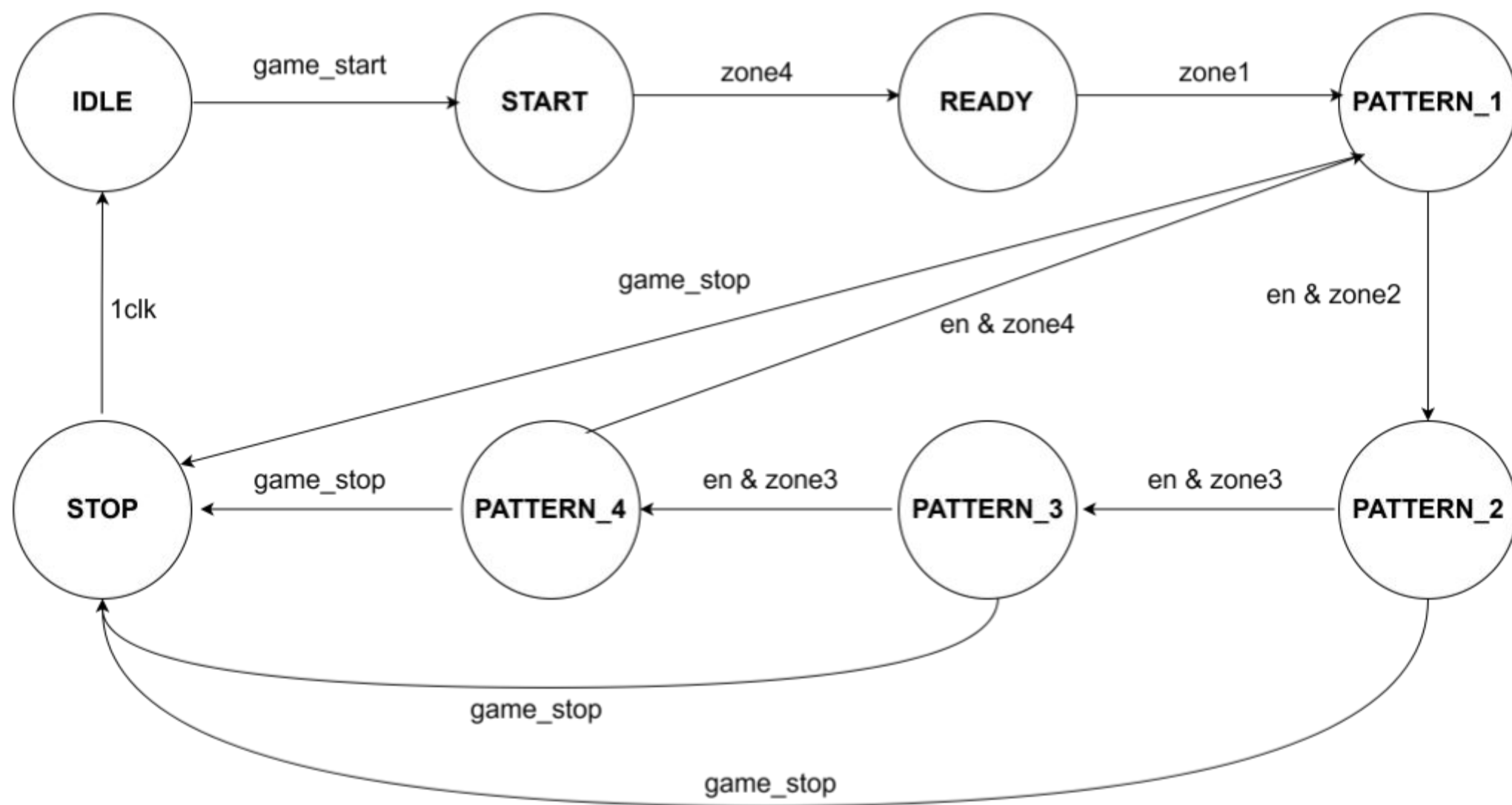
ZONE 기반 상태 전이
지휘 상태 전이 시 현재 상태, Valid, Pattern Tick 출력



지휘봉이 ① → ② → ③ → ④ 순으로
각 ZONE을 통과할 때마다 Pattern Tick 발생

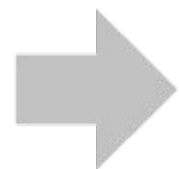
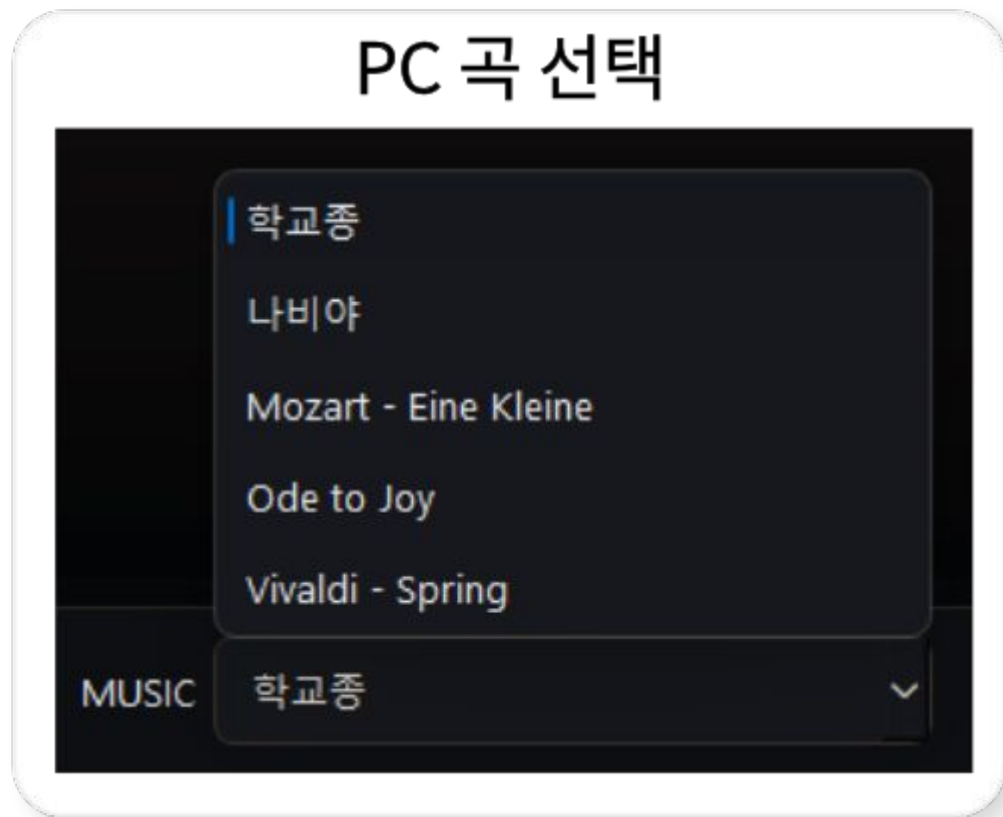
🎵 기능 구현 04 지휘 상태 판정 모듈

■ FSM



STATE	역할	핵심 동작
IDLE	초기화	Counter 초기화
START	측정 준비	zone4 도착 시 Ready
READY	지휘 시작 대기	Zone 1 → Pattern_1
PATTERN_1	지휘 state	Zone2 → Pattern_2
PATTERN_2	지휘 state	Zone3 → Pattern_3
PATTERN_3	지휘 state	Zone4 → Pattern_4
PATTERN_4	지휘 state	Zone1 → Pattern_1 Stop시에 Stop state
STOP	정지 처리	다음 clk에 IDLE State로 복귀

🎵 기능 구현 05 곡 선택 디코더 모듈



점수 계산 모듈
지휘 속도 계산 모듈

■ PC 상태 (i_pc_state)

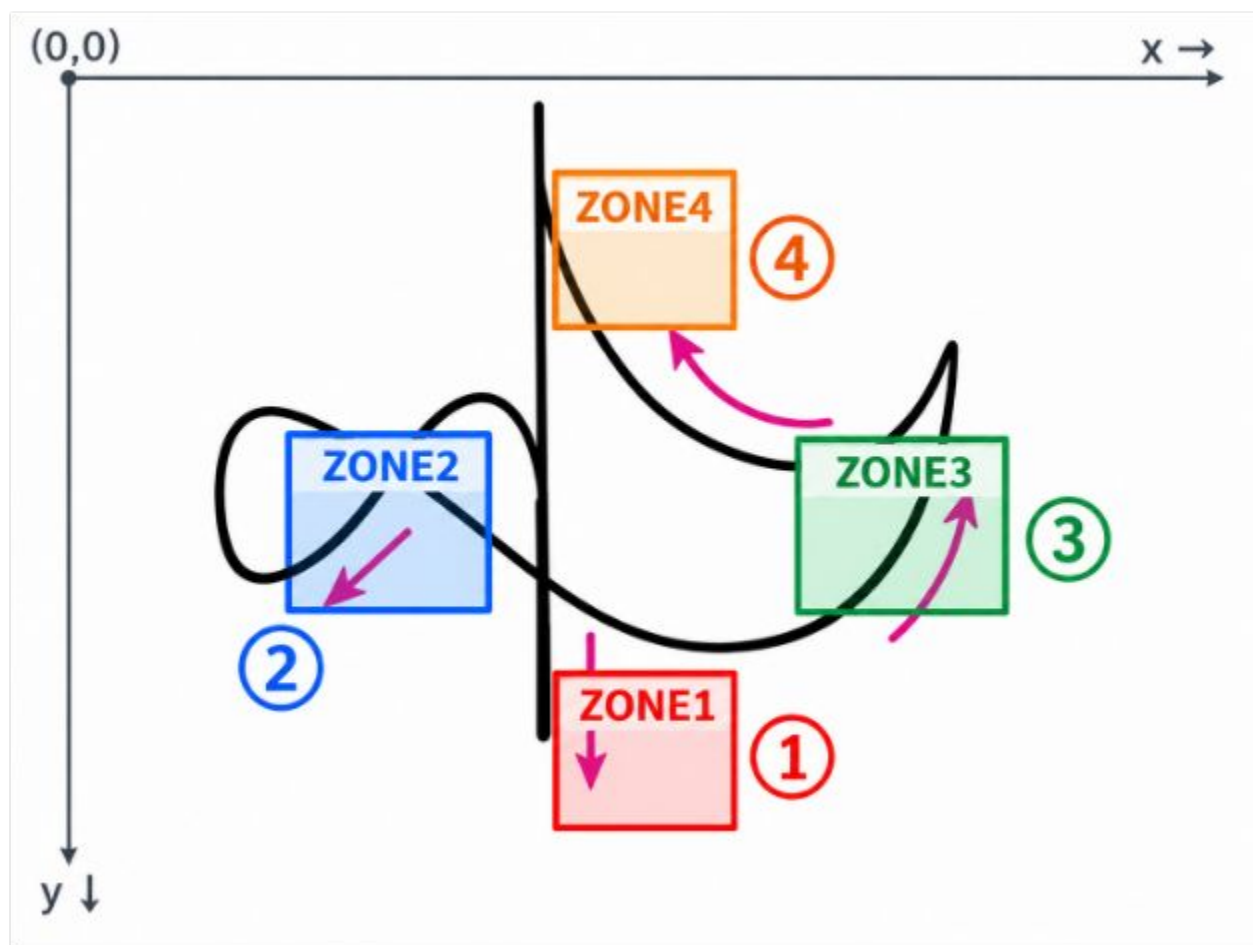
값	상태 이름	설명
000	MAIN	메인 화면
001	MENU	곡 선택 화면
010	PC_READY	곡 선택 완료 (BPM 출력)
011	PC_GAME_ING	게임 진행 중
100	PC_STOP	게임 종료

■ 곡 번호 및 BPM

값	곡 이름	BPM (o_song_bpm)
000	학교종	120
001	나비야	110
010	Mozart - Eine Kleine	145
011	Ode to Joy	100
100	Vivaldi - Spring	115

🎵 기능 구현 06 지휘 속도 계산 모듈

■ 지휘 패턴 (4/4박자)



지휘봉이 ① → ② → ③ → ④ 순으로
각 ZONE을 통과할 때 사이의 시간 측정

■ BPM(Beats Per Minute) 계산

$$60 \text{ BPM} = 60 \text{ beat} / 60\text{sec}$$

$$\Leftrightarrow \text{BPM} = 60 * 1000 / (\text{count} [\text{ms}])$$

<BPM 변환>

Pattern Tick → count[ms] → BPM → 5 BPM 단위 보정

■ 속도 반영

1) 현재 지휘 속도[bpm] > 노래 재생 속도[bpm]

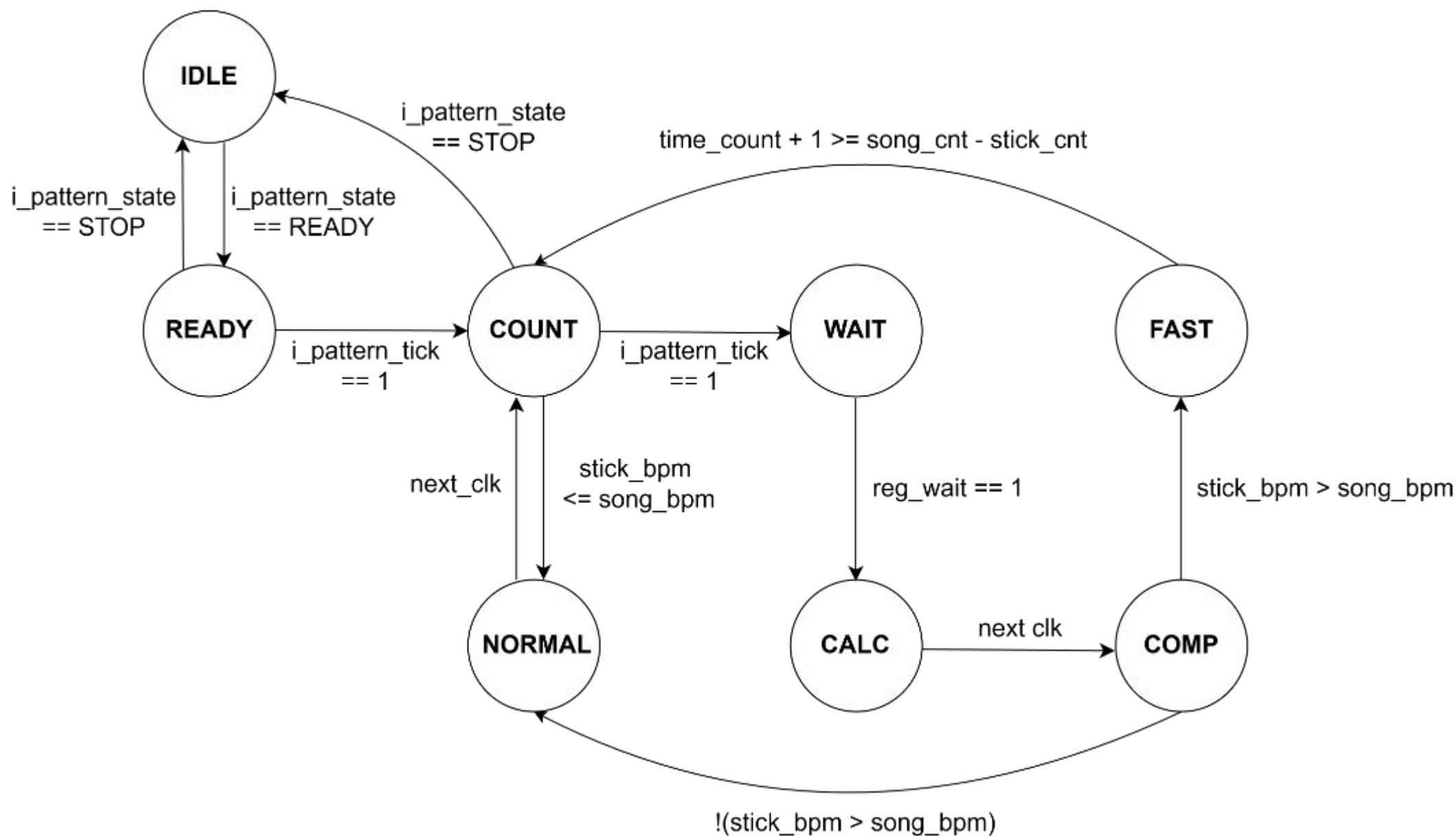
→ 남은 박자 대기 후 지휘 속도 반영

2) 현재 지휘 속도[bpm] ≤ 노래 재생 속도[bpm]

→ 즉시 지휘 속도 반영

🎵 기능 구현 06 지휘 속도 계산 모듈

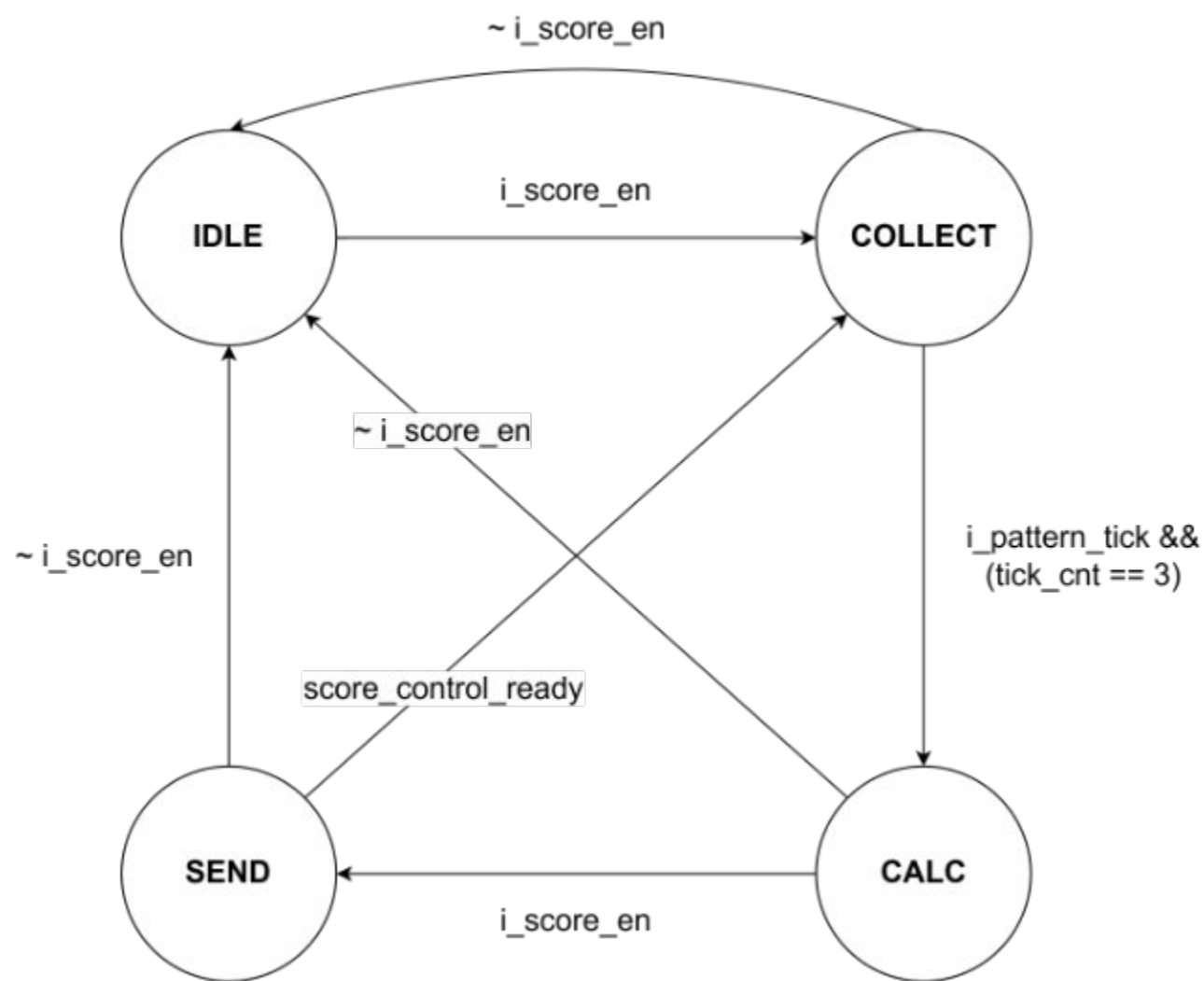
■ FSM



STATE	역할	핵심 동작
S_IDLE	초기화	Counter 초기화
S_READY	측정 준비	기존 Song BPM 설정
S_COUNT	지휘 주기 측정	ms_tick마다 Count 증가
S_WAIT	계산 대기	BPM 계산 안정화
S_CALC	BPM 계산	$60000 / \text{count}$
S_COMP	BPM 비교	Stick BPM ↔ Song BPM
S_FAST	빠른 지휘 처리	기존 박자 잔여시간 대기
S_NORMAL	일반 속도 처리	새 BPM 즉시 반영

🎵 기능 구현 07 점수 계산 모듈

■ FSM



STATE	역할	핵심 동작
IDLE	초기화	점수, RAM, RAM 시작 번호 초기값 설정
COLLECT	속도 데이터 수집	Tick마다 RAM에 속도 저장
CALC	점수 계산	4개의 속도 평균 계산 → 점수 반영
SEND	점수 전송	Valid = 1에 점수 전송

■ 개요

실시간 지휘속도를 원곡 속도와 비교하여 점수를 계산하는 모듈

■ 주요 특징

01 4개 속도 기반 평가
1/4박자마다 지휘 속도 측정 → size = 4

02 원곡 속도와 비교
4개의 속도를 평균내서 원곡 속도와 차이 계산

03 점수 변환
속도 차이에 따라 점수 부여 (+10 ~ - 10점, ±20bpm)

🎵 기능 구현 08 볼륨 계산 모듈

■ 개요

왼손의 GREEN 중심 Y좌표를 입력받아 손의 상·하 움직임을 인식하고, 음악의 볼륨을 실시간으로 조절하는 모듈

■ 주요 특징

01

기준 좌표 설정

손 검출 후 초기 Y좌표를 base_y로 저장

02

상대 Y좌표 기반 조절

위로 이동 시 볼륨 증가, 아래로 이동 시 감소

03

기준 좌표 자동 갱신

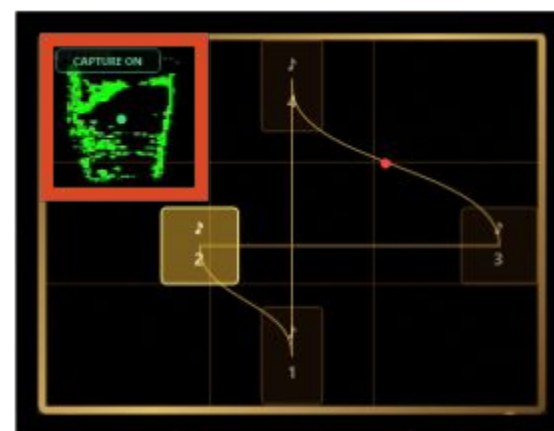
볼륨 변경 후 현재 손 위치를 새 기준점으로 설정

04

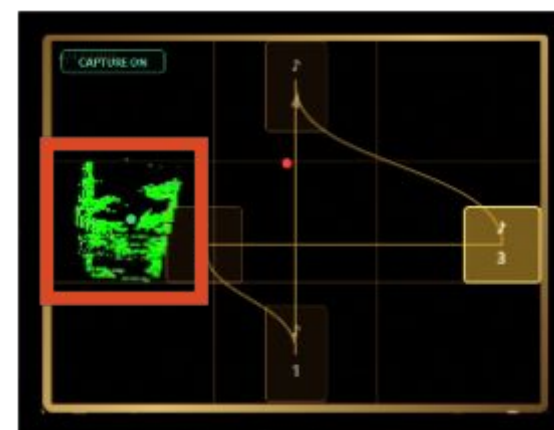
출력 범위 제한

볼륨을 0~100으로 Clamp하고 변경시에만 출력

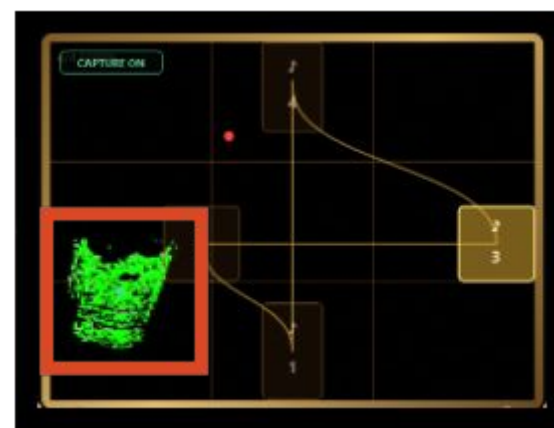
■ 동작 예시



손을 위로 이동
→ 볼륨 증가



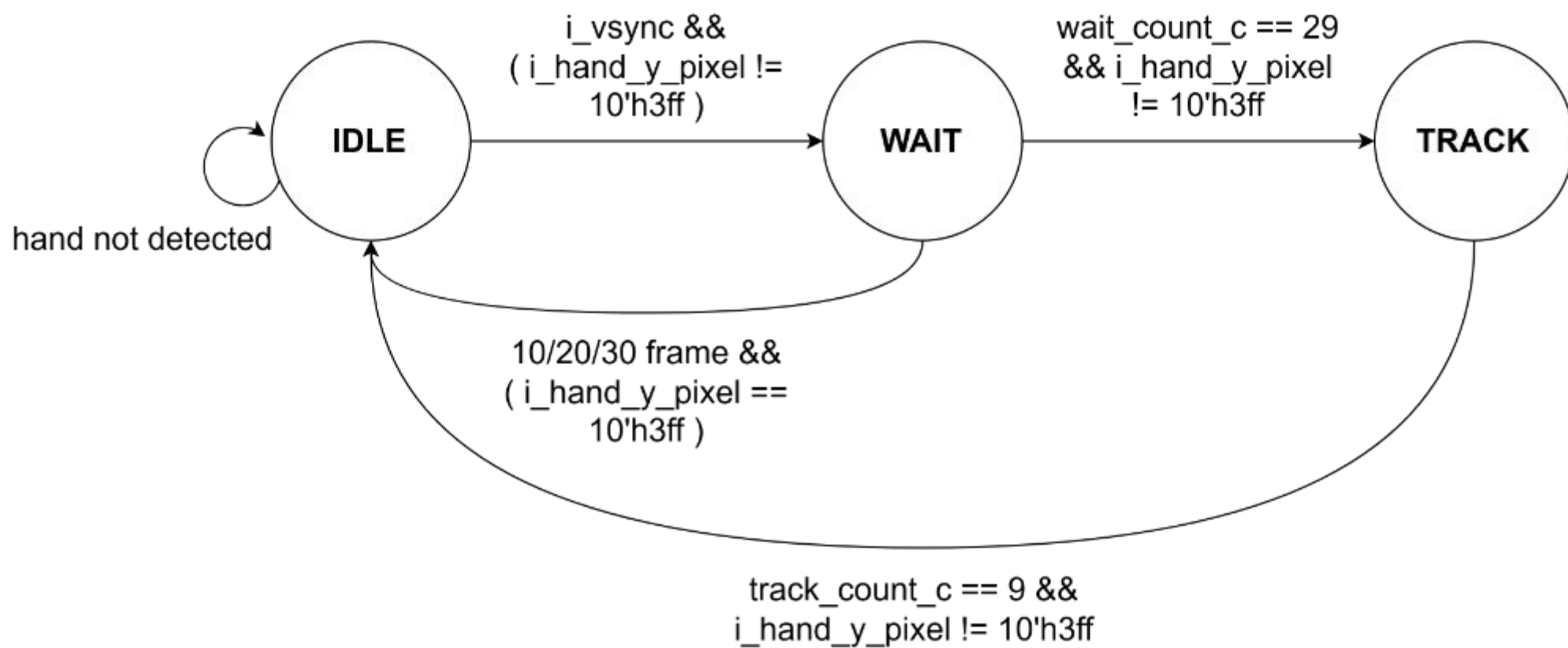
기준 위치에서
현재 볼륨 유지



손을 아래로 이동
→ 볼륨 감소

🎵 기능 구현 08 볼륨 계산 모듈

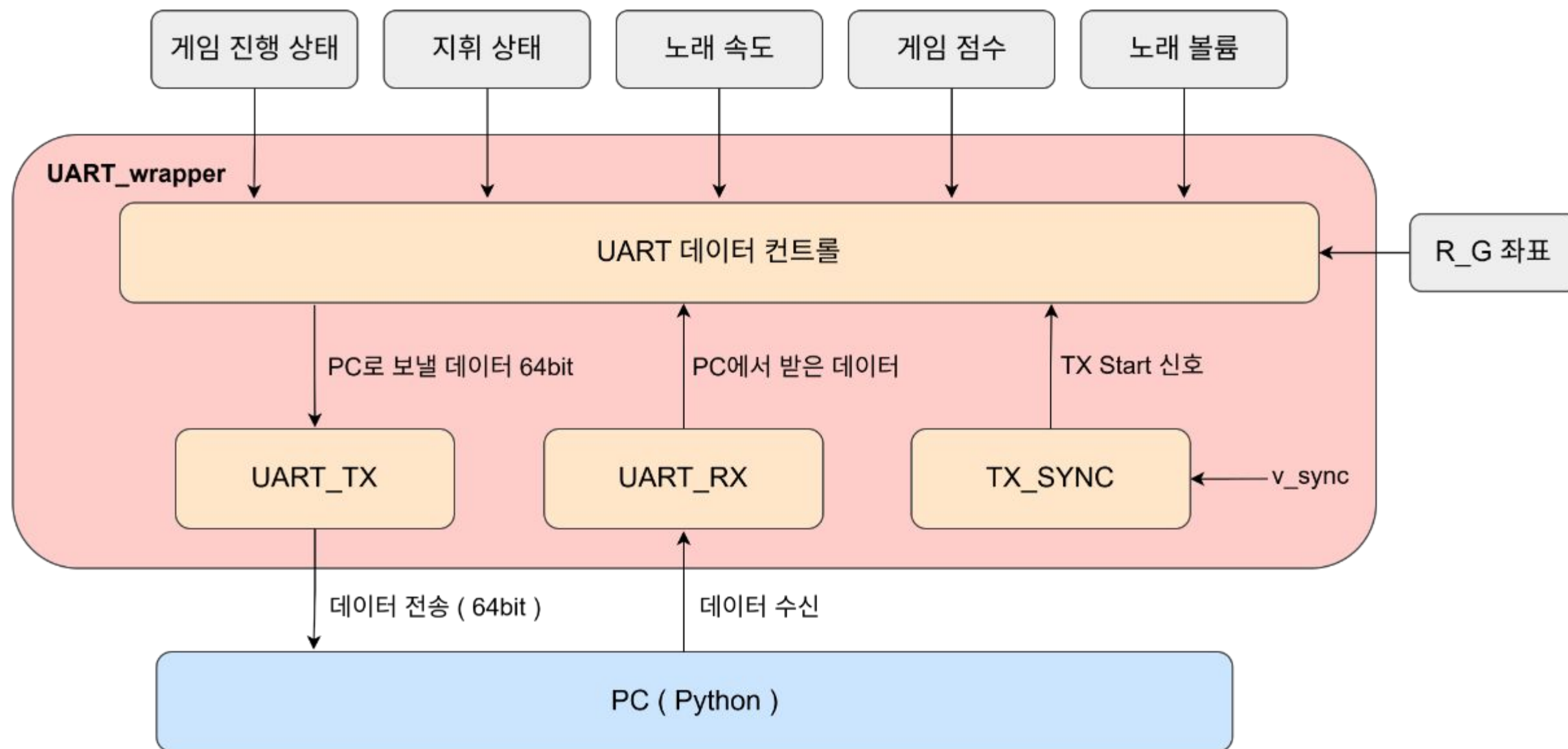
■ FSM



STATE	역할	핵심 동작
S_IDLE	손검출 대기	Counter 초기화, 유효한 손 y좌표 검출 시 WAIT 상태 진입
S_WAIT	기준 좌표 설정	30 Frame 동안 손 검출 확인 후 현재 y좌표를 base_y에 저장
S_TRACK	지휘 주기 측정	10 Frame마다 y좌표 변화량 계산, 볼륨 증감 후 base_y 갱신

🎵 통신부 | UART Wrapper

■ UART 송수신 제어와 64bit 데이터 구성을 담당하며, PC와 FPGA 간 제어 정보를 교환하는 모듈



01 UART 통신 타이밍 제어

TX_SYNC를 이용해 v_sync 기준 전송 시점 제어

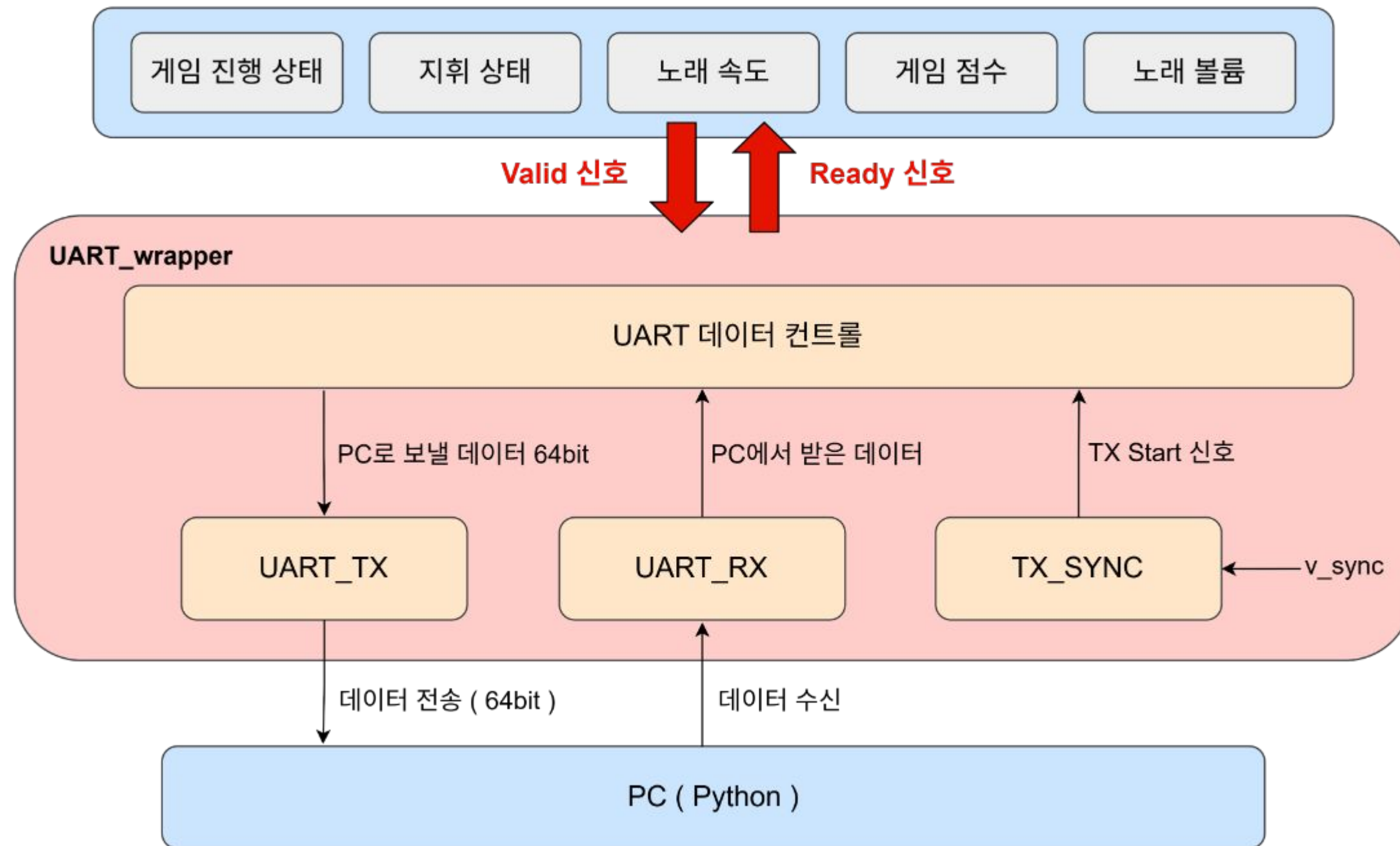
02 64bit 데이터 전송

게임 상태, 지휘 상태, BPM, 점수, 좌표 등 송신 데이터 통합

03 PC 상태 / 곡 정보 수신

PC의 상태와 선택 곡 정보를 RX로 수신

♪ 기능 구현 09 UART 기반 PC 통신



01 READY 신호
UART 데이터 컨트롤이 새 데이터를 받을 수 있는 상태임을 상위 모듈에 알림

02 VALID 신호
Game Logic에서 새로운 데이터가 준비되었을 때 데이터와 함께 VALID를 출력

∴ READY/VALID Handshake로 데이터 전달 안정성 ↑

🎵 기능 구현 09 UART 기반 PC 통신 | TX DATA

■ 64bit 송신 데이터 구성

필드	MSB	LSB	비트폭	주요 전송 데이터
GAME_FSM	63	62	2	IDLE, READY, GAMEING
XY_DEC	31	22	40	X : 0 ~ 320 Y : 0 ~ 240
PATTERN	21	19	3	READY, PATTERN 1 ~ 4
VOL	18	12	7	0 ~ 100
SPEED	11	4	8	30 ~ 220 (BPM)
SCORE	3	0	4	0 ~ 10

Handshake, 연산 지연을 고려하여 16 클럭 뒤에 일괄 전송

■ TX_SYNC 전송 타이밍

```
module sync_delay (  
    input  logic clk,  
    input  logic rst,  
    input  logic pixel_dec_sync,  
    output logic tx_sync  
);  
  
typedef enum logic {  
    IDLE = 1'b0,  
    COUNT  
} state_e;  
state_e c_state, n_state;  
logic n_enable, enable;  
  
// sync counter  
logic [3:0] cnt;  
always_ff @(posedge clk, posedge rst) begin  
    if (rst) begin  
        cnt <= 0;  
        tx_sync <= 1'b0;  
    end else begin  
        if (enable) begin  
            if (cnt == 16 - 1) begin  
                cnt <= 0;  
                tx_sync <= 1'b1;  
            end else begin  
                cnt <= cnt + 1;  
                tx_sync <= 1'b0;  
            end  
        end  
    end else begin  
        tx_sync <= 1'b0;  
    end  
end  
end
```

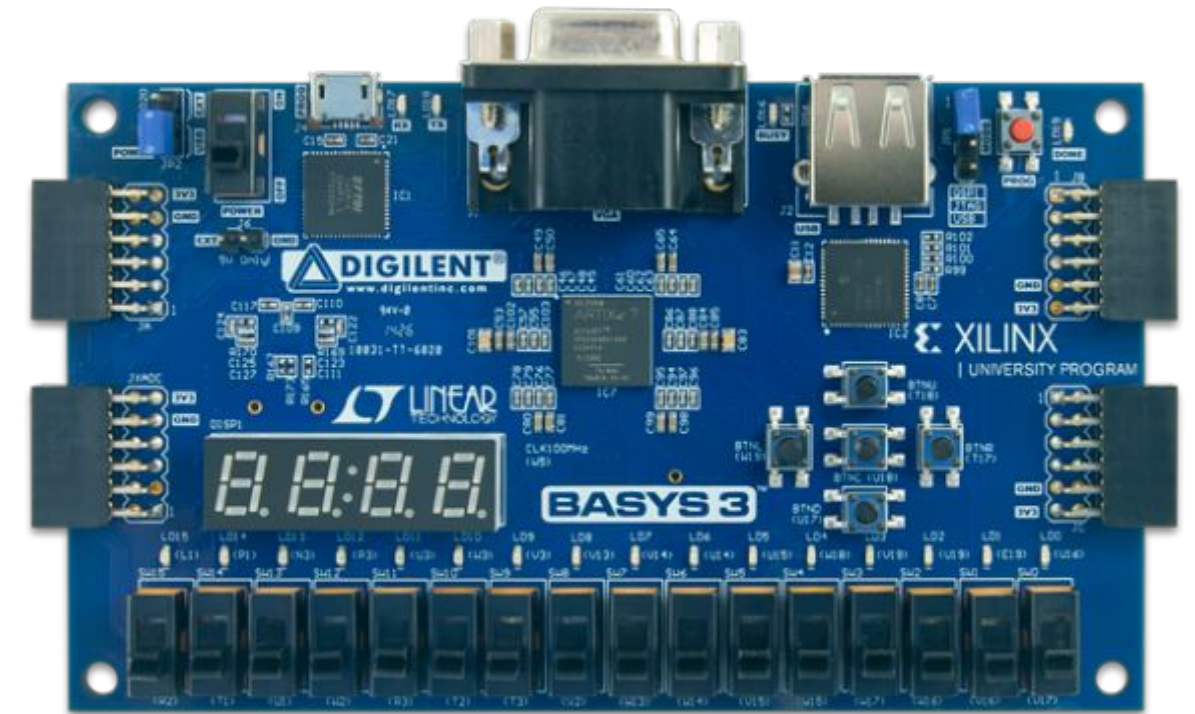
♪ 기능 구현 09 UART 기반 PC 통신 | RX DATA

■ 8bit 수신 데이터 구성

필드	MSB	LSB	비트폭	주요 전송 데이터
GAME_FSM	6	4	3	PC_IDLE, PC_MENU PC_READY, PC_START
O_PC_SONG	2	0	3	0 ~ 7 (곡 번호)



PC의 입력을 UART RX로 수신



04

UVM 검증

🎵 입력부 검증 | RG Detect

01 시나리오

No.	시나리오	설명
01	Color & XY Detection	RED / GREEN 검출 및 중심좌표 확인
02	Frame & UART Data	Frame End → VSYNC → VALID 타이밍 및 o_pixel 패킹 검증
03	Handshake & Payload Stability	Handshake 규격 검증 READY backpressure 발생시 데이터 유지 검증
04	500-Frame Regression	객체 상태·위치·READY 조건 조합 반복 검증, 500/500 PASS

단일 Regression Sequence에서 객체 상태·위치·READY 지연 조건을 변화시키며 500 Frame 검증 수행

02 Scoreboard

■ Actual vs Expected

```
.agent.driver [DRV] F1 INPUT : RED=1 GREEN=0 READY_DLY=0
.agent.monitor [MON] F1 FRAME_END : x=320 y=239
.agent.monitor [MON] F1 RESULT : VSYNC=1 Stick=(32,28) Hand=(1023,1023)
.agent.monitor [MON] F1 VALID : valid=1 ready=1
.scoreboard [SCB] F1 XY : EXP Stick=(32,28) Hand=(1023,1023) | ACT Stick=(32,28) Hand=(1023,1023)
.scoreboard [SCB] F1 PAYLOAD : EXP=0x0801cfffff | ACT=0x0801cfffff
.scoreboard [SCB] F1 TIMING : S->V=1clk V->VALID=1clk READY_DLY=0clk HS=0clk
.scoreboard [SCB] F1 RESULT : PASS
```

Compared=500 Pass=500 Errors=0

03 Coverage

■ Code Coverage

Name	Score	Line	Toggle	FSM	Condition	Branch
tb_top	76.25%	93.57%	86.41%		43.64%	81.40%
DUT	75.92%	91.89%	86.77%		43.64%	81.40%
U_RGB_FILTER	56.13%	77.78%	70.39%		9.68%	66.67%
U_XY_DETECTION	92.63%	98.67%	95.06%		87.50%	89.29%
vif	85.04%		85.04%			

∴ 각 모듈의 line/condition/branch/toggle code coverage 수집

■ Functional Coverage

Cover Group Item	Score	Instances
rg_detect_uvm_pkg::rg_dete...	100.00%	100.00%
hand_x_position_cp	100.00%	
hand_y_position_cp	100.00%	
object_state_cp	100.00%	
payload_stable_cp	100.00%	
ready_delay_cp	100.00%	
sentinel_to_vsync_cp	100.00%	
stick_x_position_cp	100.00%	
stick_y_position_cp	100.00%	
uart_pack_cp	100.00%	
valid_to_handshake_cp	100.00%	
vsync_to_valid_cp	100.00%	
object_ready_cross	100.00%	

• 객체 상태/위치 및 READY 지연 조건을 Coverpoint·Cross로 구성하여 주요 기능 시나리오 검증

Functional Coverage=100.0%

🎵 제어부 검증 | Game Logic

01 시나리오

No.	시나리오	설명
01	Conduct_fsm_test	FSM 상태 전이 검증
02	bpm_calc_test	지휘 속도 계산 모듈 검증
03	score_test	점수 계산 로직 검증
04	full_game_scenario_test	전체 게임 시나리오 검증

02 Scoreboard

Actual vs Expected

```
.agent.driver [DRV] 다음 입력 : stick=(160,200), hand_y=200, pc_state=3, song=2, frame_gap=15
.agent.monitor [MON] Pattern 상태 변화 : 5 -> 6, tick=1
.scoreboard [SCB] BPM 비교 : expected=120, actual=120
.agent.monitor [MON] BPM 출력 : 120
.scoreboard [SCB] Score 확인 : previous=6, actual=7
.agent.monitor [MON] Score 출력 : 7
.driver [DRV] 다음 입력 : stick=(220,135), hand_y=100, pc_state=3, song=1, frame_gap=1
.monitor [MON] Volume 출력 : 50
```

```
.scoreboard [SCB] 최종 결과 : Total=657780, PASS=657780, FAIL=0
```

03 Coverage

Code Coverage

Name	Score	Line	Toggle	FSM	Condition	Branch
tb_top	83.47%	97.35%	77.47%	74.42%	78.90%	89.21%
dut	83.47%	97.35%	77.47%	74.42%	78.90%	89.21%
u_game_fsm	94.86%	100.00%	92.50%	100.00%	90.91%	90.91%
u_game_score	82.58%	94.74%	85.29%	71.43%	83.33%	78.12%
u_pattern_stick	79.45%	96.30%	87.21%	61.11%	59.09%	93.55%
u_song_decoder	96.71%	100.00%	86.84%		100.00%	100.00%
u_speed_calc	83.74%	98.77%	56.01%	90.00%	85.71%	88.24%
u_vol_control	88.83%	97.06%	95.96%	75.00%	80.49%	95.65%

∴ 각 모듈의 line/condition/branch/FSM/toggle code coverage 수집

주요 Coverpoint

Group	Score	Instances	U+C
game_logic_pkc::gam...	98.96%	98.96%	48
cp_bpm	100.00%		3
cp_game_state	100.00%		3
cp_game_transition	100.00%		4
cp_pattern_state	100.00%		8
cp_pattern_transition	91.67%	12	
cp_score	100.00%	5	
cp_song	100.00%	8	
cp_volume	100.00%	5	
tb_top.me.obj.gam...	98.96%		48

Corner case 추가한 full_case_coverage_test

```
[COV] Functional Coverage = 98.96%
d [SCB] 최종 결과 : Total=2405795, P
```

98.96%

🎵 통신부 검증 | UART Wrapper

01 시나리오

No.	시나리오	설명
01	Random	다양한 입력에서 송수신 데이터 비교
02	Rx_Corner	0x00, 0x55, 0xAA, 0xFF 확인
03	Tx Field	Tx data field별 활성화 random 체크
04	Full Duplex	UART 기능 검증, Tx/Rx 동시 송수신 위주 검증

02 Scoreboard

■ Actual vs Expected

테스트	TX PASS	RX PASS	FAIL
Random	1,500	300	0
RX Pattern	0	12	0
TX Field	189	0	0
Full Duplex	15	120	0

03 Coverage

■ Code Coverage

Name	Score	Line	Toggle	FSM	Condition	Branch	Assert
tb_top	89.55%	95.33%	98.42%	66.67%	87.13%	89.74%	100.00%
dut	87.35%	94.87%	98.34%	66.67%	87.13%	89.74%	
U_DATA_CONTROL	91.61%	100.00%	99.78%		66.67%	100.00%	
U_SYNC_DELAY	96.69%	100.00%	95.83%		100.00%	90.91%	
U_UART	87.14%	93.30%	96.72%	66.67%	92.11%	86.90%	
vif	99.74%		99.49%				100.00%

∴ 각 모듈의 line/condition/branch/FSM/toggle/ Assert coverage 수집

■ Functional Coverage

Group	Score	Instances	U+C	U	C	X	Goal
uart_wrapper_pkg::uart_wrapper_coverage::handshake_cg	100.00%	100.00%	76	0	76	0	100%
uart_wrapper_pkg::uart_wrapper_coverage::overlap_cg	100.00%	100.00%	1	0	1	0	100%
uart_wrapper_pkg::uart_wrapper_coverage::rx_cg	100.00%	100.00%	7	0	7	0	100%
uart_wrapper_pkg::uart_wrapper_coverage::tx_cg	100.00%	100.00%	399	0	399	0	100%

■ Assertion Coverage

Assert	Gen Info	Type	Attempt	Success/V	Failure
tb_top.vif.a_game_valid_ready_deassert		Assertion	...931223	908	0
tb_top.vif.a_pattern_state_valid_deassert		Assertion	...931223	931	0
tb_top.vif.a_pixel_valid_deassert		Assertion	...931223	908	0
tb_top.vif.a_score_valid_deassert		Assertion	...931223	905	0
tb_top.vif.a_speed_valid_deassert		Assertion	...931223	876	0
tb_top.vif.a_volume_valid_deassert		Assertion	...931223	890	0

05

구현 결과

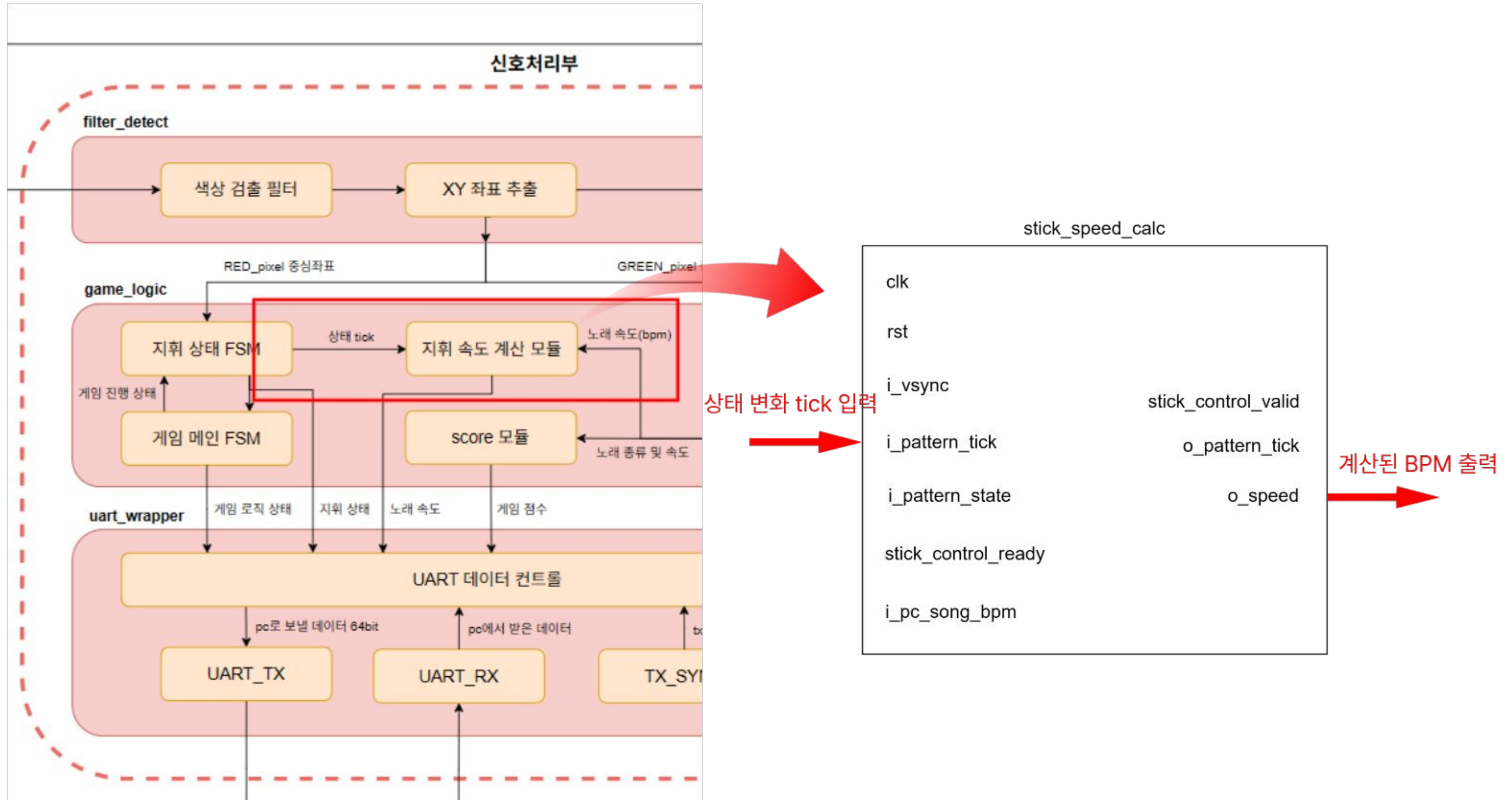
♪ 구현 영상



06

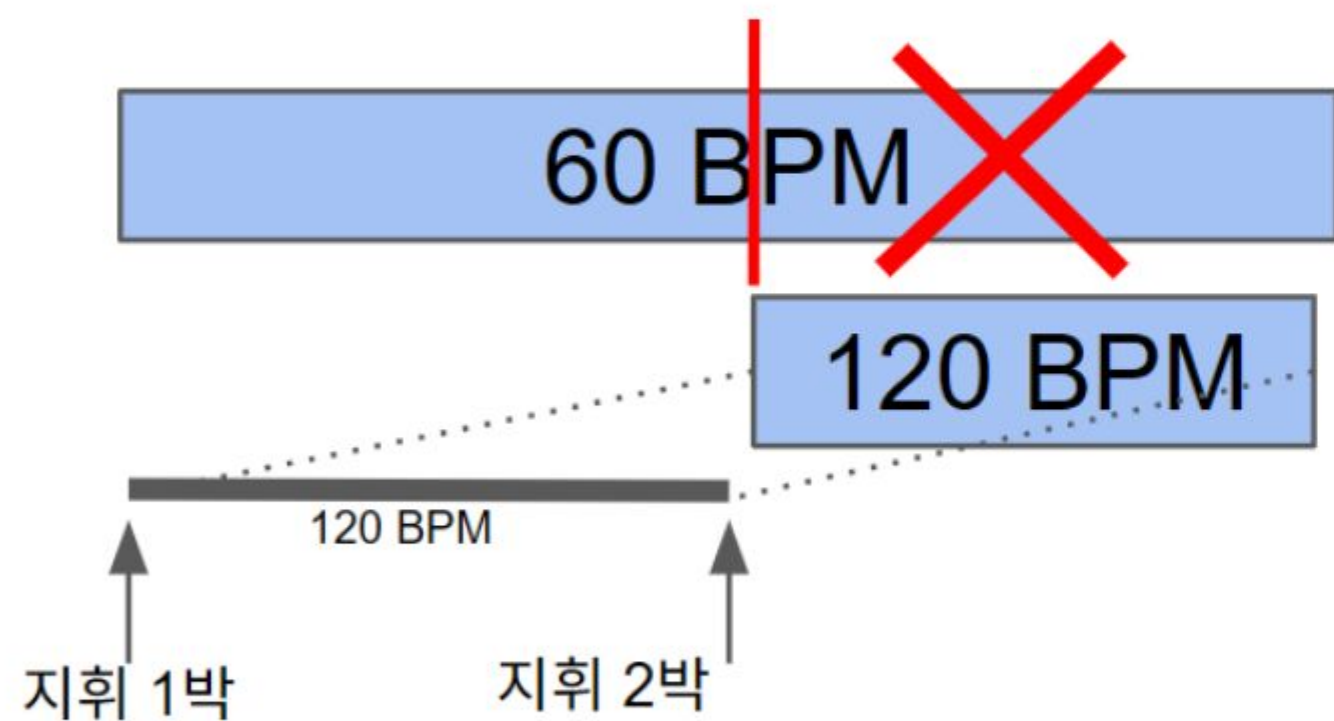
트러블 슈팅

♪ 트러블 슈팅 01 문제 상황 | 음원 손실

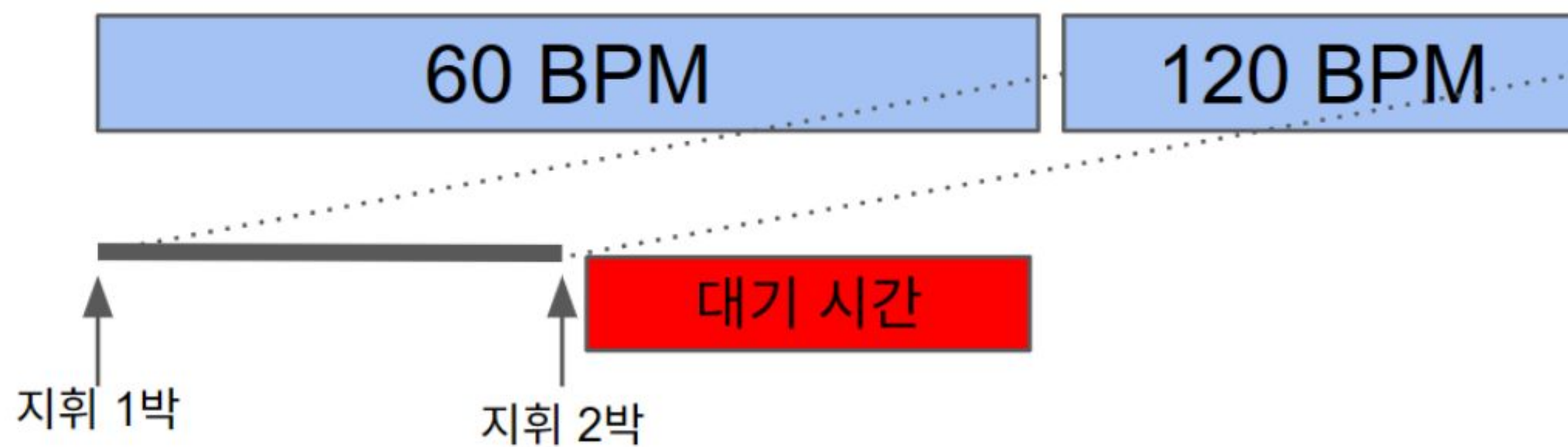


🎵 트러블 슈팅 01 문제 상황 | 음원 손실

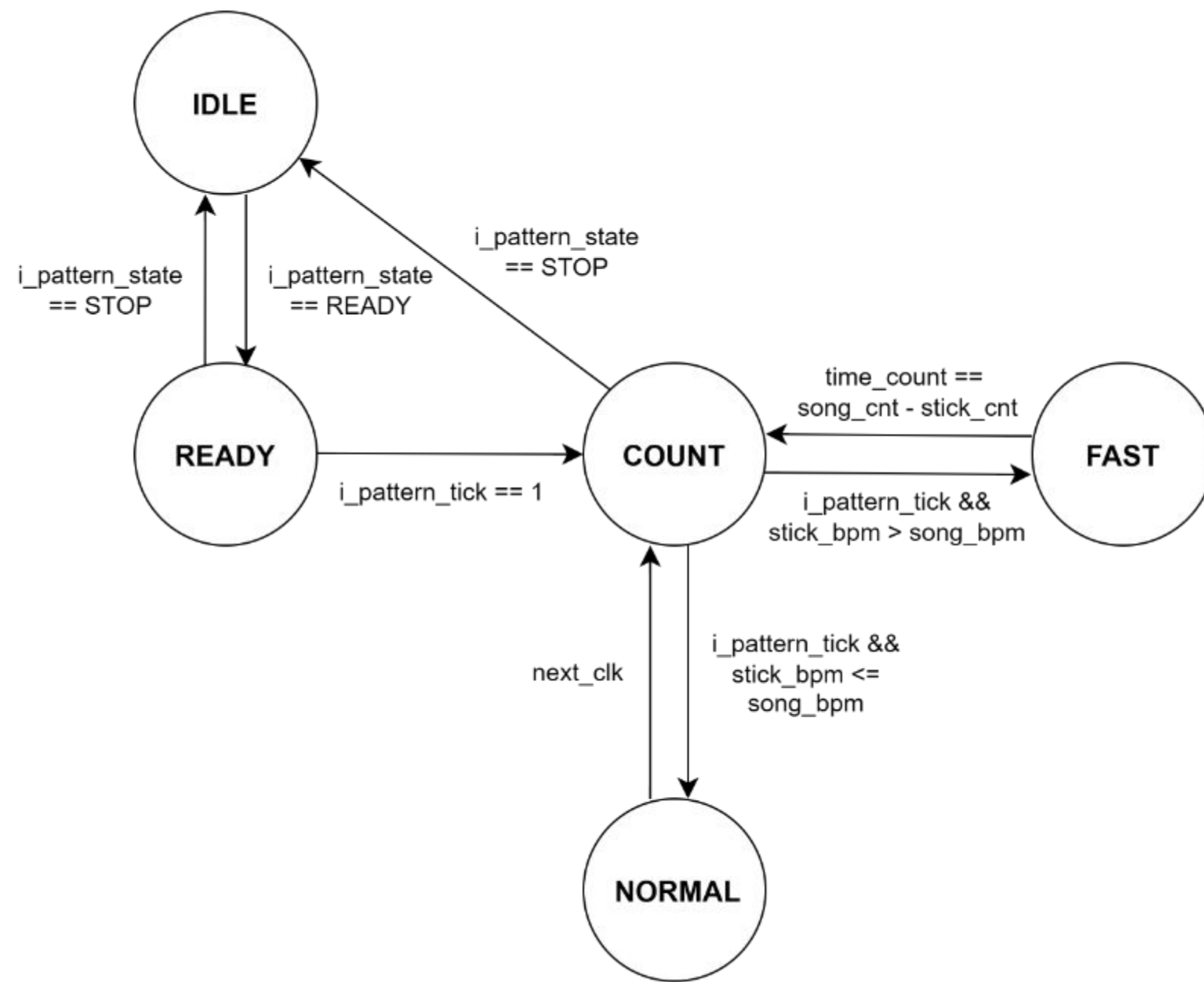
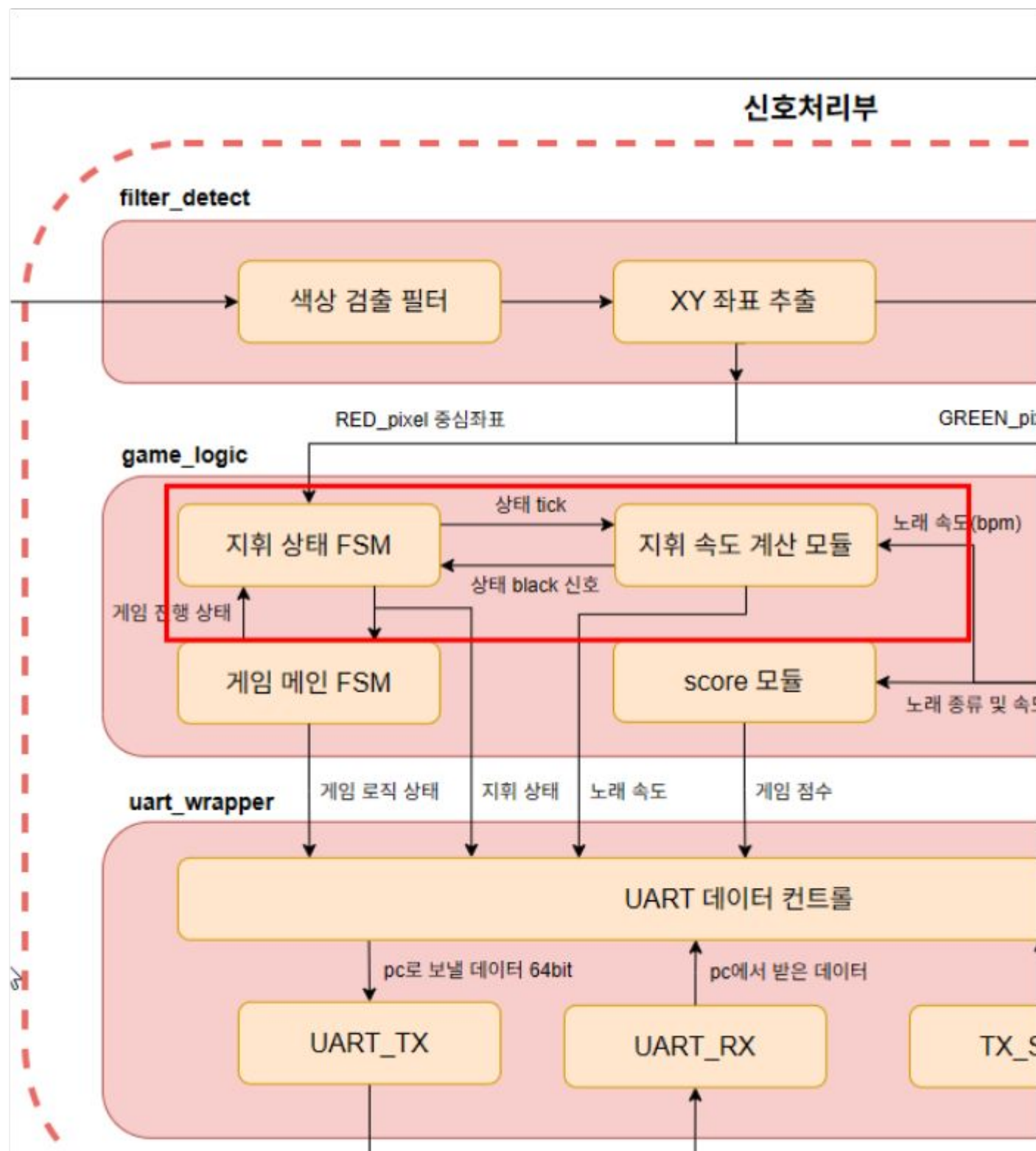
Predicted



Expected



🎵 트러블 슈팅 02 해결 과정 | FSM 도입



🎵 트러블 슈팅 02 해결 과정 | Negative Slack 발생과 해결

Tcl ConsoleMessagesLogReportsDesign RunsPowerDRCMethodologyTiming x

Q

≡

⬇

!

Q

—

↶

⬢

⌚

!

Intra-Clock Paths - sys_clk_pin - Setup

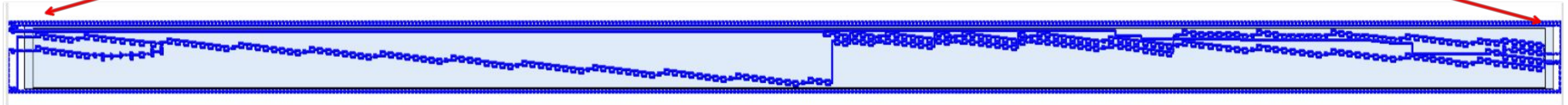
unconstrained_internal_endpoints (831)

no_input_delay (17)

no_output_delay (18)

multiple_clock (0)

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
↳ Path 1	-80.719	302	48	U_GAME_LOGIC/u_speed_.../count_reg_reg[8]/C	U_GAME_LOGIC/u_spe..._c_state_reg[2]/D	90.616	60.860	29.756	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 2	-80.665	302	48	U_GAME_LOGIC/u_speed_.../count_reg_reg[8]/C	U_GAME_LOGIC/u_spe..._c_state_reg[0]/D	90.561	60.860	29.701	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 3	-80.659	302	48	U_GAME_LOGIC/u_speed_.../count_reg_reg[8]/C	U_GAME_LOGIC/u_spe..._c_state_reg[1]/D	90.558	60.860	29.698	10.0	sys_clk_pin	sys_clk_pin		0.035



-80.719ns Negative Slack 발생

🎵 트러블 슈팅 02 해결 과정 | 조합경로 경량화

01 bpm → cnt 나눗셈 LookUpTable 적용

```
integer j;
logic [10:0] bpm_to_cnt_LUT[30:221];

initial begin
    for (j = 30; j <= 220; j = j + 5) begin
        bpm_to_cnt_LUT[j] = (16'd60 * MS_FREQ) / j;
    end
end

end

if (n_state == S_COUNT) begin
    // 다음 음악 기준값 갱신
    song_bpm    <= stick_bpm;
    song_cnt    <= bpm_to_cnt_LUT[stick_bpm]; // LUT
    time_count <= 11'd0;
end else if (ms_tick) begin
    time_count <= time_count + 11'd1;
end
```

02 10ns → 1ms tick 카운팅

```
ms_tick_gen #(
    .CLK_FREQ(CLK_FREQ),
    .MS_FREQ (MS_FREQ)
) u_ms_tick_gen (
    .clk(clk),
    .rst(rst),
    .en(ms_en),
    .ms_tick(ms_tick)
);
```

Timing													
Intra-Clock Paths - sys_clk_pin - Setup													
Name	Slack ^1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
↳ Path 1	-10.176	41	24	count_reg_reg[1]/C	reg_bpm_comp_reg_rep[0]/D	20.283	12.131	8.152	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 2	-9.636	41	24	count_reg_reg[1]/C	reg_bpm_comp_reg[0]/D	19.790	12.131	7.659	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 3	-8.571	37	24	count_reg_reg[1]/C	reg_bpm_comp_reg_rep[1]/D	18.276	11.011	7.265	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 4	-8.037	37	24	count_reg_reg[1]/C	reg_bpm_comp_reg[1]/D	18.021	11.011	7.010	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 5	-6.686	32	24	count_reg_reg[1]/C	reg_bpm_comp_reg_rep[2]/D	16.294	9.716	6.578	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 6	-6.164	32	24	count_reg_reg[1]/C	reg_bpm_comp_reg[2]/D	16.050	9.716	6.334	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 7	-4.505	27	24	count_reg_reg[1]/C	reg_bpm_comp_reg_rep[3]/D	14.161	8.272	5.889	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 8	-3.956	27	24	count_reg_reg[1]/C	reg_bpm_comp_reg[3]/D	13.920	8.272	5.648	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 9	-2.422	23	24	count_reg_reg[1]/C	reg_bpm_comp_reg_rep[4]/D	12.078	6.922	5.156	10.0	sys_clk_pin	sys_clk_pin		0.035
↳ Path 10	-1.859	23	24	count_reg_reg[1]/C	reg_bpm_comp_reg[4]/D	11.823	6.922	4.901	10.0	sys_clk_pin	sys_clk_pin		0.035

🎵 트러블 슈팅 02 해결 과정 | Multi Cycle Path 지정

03 나누기 연산 지연

```
end
S_WAIT: begin
    reg_wait <= ~reg_wait;
end
S_CALC: begin
    reg_bpm_comp <= bpm_idx;
    count_reg <= 11'd0;
end
S_COMP: begin
    stick_bpm <= arr[reg_bpm_comp];
end
// FAST
// 이전 음악이 끝날 때까지 대기
S_FAST: begin
    // 지휘 count는 하지 않음
    count_reg <= 11'd0;
```

04 multi cycle path 지정

```
set_multicycle_path 3 -setup -from [get_pins {U_GAME_LOGIC/u_speed_calc/count_reg_reg[*]/C}] -to [get_pins {U_GAME_LOGIC/u_speed_calc/reg_bpm_comp_reg[*]/D}]
set_multicycle_path 2 -hold -from [get_pins {U_GAME_LOGIC/u_speed_calc/count_reg_reg[*]/C}] -to [get_pins {U_GAME_LOGIC/u_speed_calc/reg_bpm_comp_reg[*]/D}]
```

Tcl Console	Messages	Log	Reports	Design Runs	Power	DRC	Methodology	Timing	x
Q [Z] [N] [O]									
Design Timing Summary									
General Information									
Timer Settings									
Design Timing Summary									
Clock Summary (1)									
> Check Timing (991)									
> Intra-Clock Paths									
Inter-Clock Paths									
> Other Path Groups									
Setup									
Worst Negative Slack (WNS): 0.205 ns									
Total Negative Slack (TNS): 0.000 ns									
Number of Failing Endpoints: 0									
Total Number of Endpoints: 2014									
Hold									
Worst Hold Slack (WHS): 0.027 ns									
Total Hold Slack (THS): 0.000 ns									
Number of Failing Endpoints: 0									
Total Number of Endpoints: 2014									
Pulse Width									
Worst Pulse Width Slack (WPWS): 4.500 ns									
Total Pulse Width Negative Slack (TPWS): 0.000 ns									
Number of Failing Endpoints: 0									
Total Number of Endpoints: 861									
All user specified timing constraints are met.									

07

느낀 점

♪ 느낀점

01 이영현 | 팀장, 아키텍처 설계 및 통합

- 여러 팀원이 각자 설계한 모듈을 하나의 시스템으로 통합하는 과정에서, 인터페이스 타이밍과 신호 규약을 사전에 명확히 정의하는 것이 얼마나 중요한지 체감했다.
- 개별 모듈은 문제없이 동작해도 통합 시점에 예상치 못한 타이밍 이슈가 발생할 수 있음을 배웠다.

02 유지민 | 지휘 속도 계산 모듈

- 현재 지휘 속도와 원곡 속도를 비교해 즉시 반영하거나 남은 박자를 기다리는 로직을 설계하면서 제어 흐름을 설계하는 것이 중요하다고 느꼈다.
- 문제를 빠르게 공유하고 서로의 설계 의도를 이해하는 협업 과정이 시스템 완성도를 높이는 핵심이라는 점을 체감했다.

03 이민 | 지휘 상태, 점수 계산 모듈 설계 및 제어부 UVM 검증

- UVM 검증을 통해 모듈 간 핸드셰이크의 동작 원리와 데이터 전달 방법에 대한 이해를 넓혔다.
- 기존의 개인·2인 단위 작업에서 팀 단위로 Verilog를 설계하면서, 모듈 간 인터페이스를 사전에 정의하고 동작 조건을 팀원들과 공유하는 것이 중요하다고 느꼈다.

04 이승열 | xy_detect, song_decoder, FPGA - PC 인터페이스 설계

- 한 프레임 내에서 좌표 계산과 데이터 전송을 완료하기 위해 o_vsync를 기준으로 모듈 동작을 동기화하며 타이밍 설계의 중요성을 배웠다.
- Git을 활용하면서 형상관리의 중요성을 느꼈고, 버전 관리 과정의 어려움과 필요성을 경험했다.

05

조정민

| 볼륨 제어 모듈 설계

- 각자 맡은 기능을 구현한 뒤 하나의 시스템으로 통합하면서, 모듈 간 인터페이스와 데이터 흐름을 맞추는 것이 중요하다는 것을 느꼈다.
- 동작 조건과 예외 상황을 먼저 정의하고, 이를 FSM과 제어 흐름으로 설계하는 과정의 중요성을 배웠다.
- 여러 의견을 조율해 하나의 방향으로 결정하는 과정에서, 기술 구현뿐 아니라 의사결정 과정도 중요하다는 점을 배웠다.

06

지승배

| 통신부 설계 및 검증

- 팀 단위의 rule, 설계 규격을 정하고 설계를 시작해 오류 원인을 좁히는 것이 수월했으며 통합이 수월했던 것 같다.
- 중간에 바뀌는 규격이 존재해도 지속적인 소통을 통해 맞춰 나갈 수 있었다.
- Negative Slack을 해결하며 성능과 속도간의 trade-off를 체감했다.
- UVM 검증을 진행하며 단일 구조에서 test, sequence를 달리하며 재사용 가능한 구조를 만드는 경험을 했다.

07

한승훈

| 입력부 설계 및 검증

- 카메라 색상 노이즈 문제를 해결하기 위해 실제 화면을 보며 Threshold와 Margin 값을 조정했고, 환경에 맞는 튜닝의 중요성을 배웠다.
- 모듈 단위로 나누는 것보다, 실제 시스템에서 신호가 언제 생성되고 어떻게 다음 블록으로 넘어가는지 맞추는 통합 과정이 더 어렵고 중요하다는 걸 느꼈다.
- Git Branch 기반으로 역할을 나누고 형상관리를 진행하면서 협업과 코드 관리의 중요성을 경험했다.



Thank You